ARES：面向 FPGA 的 RISC-V 单发射流水线 SoC

摘要

ARES 是以 Mundus 32 位顺序单发射处理器为核心的嵌入式 SoC，主线 FPGA 目标为盘古 200Pro+ 的 PG2L200H-6IFBB676。处理器实现 RV32I 整数数据通路、RV32M、Zicsr、ECALL/MRET、机器定时器中断及 custom-0 CRC8；FENCE 用于访存排空，EBREAK 在退休后停机，FENCE.I 尚无指令缓存失效路径。流水线由取指请求、取指响应、ID、ISS、EX1–EX4 和 WB 共九级组成；取指侧使用请求/响应握手、两项在途元数据队列和八项前端队列，取指延迟随 Cache 命中与回填变化。默认配置采用 16 项 Bimodal/BTB、8 项 RAS、四路 ICache、8 KiB 两路 DCache，以及外设桥中的四项写队列。SoC 提供单核和 2/4-hart AMP 配置，并集成矩阵、状态机和链表 MMIO 加速器；共享 L2 另属实验配置。单核 RT-Thread Nano 与 MSH 使用独立固件。验证入口包括 Verilator 程序回归、模块定向测试及推断 BRAM/UART 板级仿真，Pango 构建使用 PDS 流程。在 100 MHz 仿真时钟下，未修改的 CoreMark 基线为 2.235012 CoreMark/MHz；以 CRC8 指令和三类引擎替换热点函数后，单核全加速为 13.384075 CoreMark/MHz，四核全加速 MULTITHREAD 聚合为 56.029283 CoreMark/MHz。

作品同时集成自研 miniTensor INT8 协处理器：4×4 阵列、行式 UB、向量单元与定点 epilogue，把卷积和 GEMV 统一到同一条命令流。应用侧以 Qwen2.5-0.5B-Instruct 构建语料、把能力压缩到 152,056 B 的 H112，char-GRU 由 17 提升到 103 char/s；手势 287 帧 INT8 100%、板端 8/8，并驱动 Three.js 全息页面。

目录

第一部分 作品概述 **3**
1.1 功能与特性 . . . 3
1.2 应用领域 . . . 3
1.3 主要技术特点 . . . 3
1.4 主要配置 . . . 4
1.5 主要创新点 . . . 4
1.6 设计流程 . . . 5

第二部分 系统组成及功能说明 **6**
2.1 整体介绍 . . . 6
2.2 硬件系统介绍 . . . 7
2.3 软件系统介绍 . . . 37
2.4 系统特色：加速器、多核协作与实时调度 . . . 41

第三部分 完成情况及性能参数 **50**
3.1 整体介绍 . . . 50
3.2 工程成果 . . . 50
3.3 ARES 验证与性能结果 . . . 51
3.4 应用与加速器特性成果 . . . 55

第四部分 总结 **56**
4.1 可扩展之处 . . . 56
4.2 心得体会 . . . 56

第五部分 参考文献 **57**

附录 A 复现信息 **59**
A.1 版本与结果记录 . . . 59
A.2 复测命令 . . . 59

附录 B 旧版 Mundus 验证资料与历史性能 **60**
B.1 仿真结果与分析 . . . 60
B.2 FPGA 验证 . . . 65
B.3 FPGA 验证结果及分析 . . . 66
B.4 IO 模拟与板级运行一致性 . . . 68

附录 C 视频采集与手势控制集成方案 **72**
C.1 OV5640 摄像头视频通路 . . . 72
C.2 手势识别与网页演示控制 . . . 73

附录 D 应用与加速器复现信息 **74**

第一部分　作品概述

1.1　功能与特性

处理器支持 RV32I 整数运算、移位、比较、访存和控制流，扩展 RV32M 乘除法、Zicsr 寄存器读改写及基础机器模式异常处理。核心按顺序单发射、单提交运行，配合重定向、数据旁路和互锁处理相关性，并保留 DCache miss replay 作为保护路径。IROM 取指和 PERIP 数据接口均具有握手控制；SoC 集成片上存储、UART、计时器、显示 IO 和 CoreMark 加速器。AMP 提供私有数据窗口与共享 mailbox，RT-Thread Nano 支持单核运行。

在处理器之外，作品给出两条应用路径：经 UART 的端侧对话与知识问答，以及 OV5640 手势识别；两者共用同一套 miniTensor、驱动与量化工具链，并驱动浏览器全息演示。

1.2　应用领域

作品面向三类场景。一是嵌入式控制：单核配合机器定时器中断运行 RT-Thread Nano，提供时间片抢占、延时唤醒和串口命令行，可作为小型实时控制器的处理器平台。二是专用计算加速：CRC8 自定义指令与矩阵、状态机、链表 MMIO 引擎给出“热点函数替换、保留软件参考实现”的加速方法，可迁移到校验、滤波、协议解析等固定算法。

三是端侧智能交互：无操作系统、无外部加速卡即可完成小模型对话、知识问答与手势控制，可用于展陈互动与无障碍辅助。

1.3　主要技术特点

前端将预测表读取流水化，提前查询候选下一 PC，并保存停顿期间的预测及取指元数据。后端以温度计码统一表达生产者结果时刻和消费者需求时刻，选择最新且就绪的旁路结果。AGU 在 ISS 提前计算访存地址，三阶段 LSU 区分 DCache 命中快路径和外部返回慢路径，外设桥用四项写队列及逐字节转发协调访存。乘法器采用四级部分积流水，除法器采用 radix-4 迭代 RTL；存储包装使用可综合 BRAM。CRC8 指令与三类 MMIO 引擎支持软硬件协同加速。

协处理器侧以 4×4 INT8 阵列、128 bit 行式统一缓冲、向量单元与定点 epilogue，把卷积和 GEMV 统一为同一条 `GEMM` 命令流。

1.4 主要配置

以下列出 RTL 默认配置及板级时钟目标。

项目	配置
处理器与扩展	Mundus，顺序单发射；RV32IM、Zicsr、基础机器模式与 CRC8
FPGA / 工具链	PG2L200H-6IFBB676 / Pango PDS
板级时钟	27 MHz 输入，PLL 配置 CPU 100 MHz / 外设 50 MHz；PDS 实现结果见第 3.3.5 节
预测器与前端	16 项 BTB、2 bit Bimodal、8 项 RAS；8 项前端队列
存储层次	单核默认 4 KiB 四路 ICache、8 KiB 两路 DCache
系统配置	单核；2/4-hart 私有存储 AMP；共享 L2 实验版

除处理器配置外，两个应用与 miniTensor 的主要指标如下。

表 1: 应用与加速器主要性能指标

项目	指标	口径
miniTensor	4×4 INT8、32 bit 累加；UB 256×128 bit	16 PE，8–10 周期/GEMM
手势帧	1,936 条命令；约 1.31 M 周期	100 MHz 仿真 13.1 ms；27 MHz 板级 48.5 ms
H112	140,742 参数 / 152,056 B	20 token/s 输出节流
A0	17 char/s（CPU）/ 103 char/s（miniTensor）	0/40 golden
手势	287 帧 INT8 100%；板端 8/8；10 Hz	3 帧确认，约 0.2 s 更新
b5 整机	CLMA 35%、DRM 63%、APM 9%	27 MHz WNS +12.85 ns

1.5 主要创新点

1. 提前查询与预测保持隐藏 BPU 读表延迟，维持每拍一次取指。
2. 温度计码统一生产与消费时机，配合两级旁路和互锁减少误停顿。
3. ISS 级 AGU、三段 LSU 与逐字节转发写队列兼顾频率和访存效率。
4. CRC8 指令与三类 MMIO 引擎替换 CoreMark 热点，单核约 6 倍加速。
5. 以 CV-X-IF 兼容命令把卷积与 GEMV 统一为同一条 `GEMM` 指令流，`BURST/GEMM2D` 把软件循环搬进硬件。
6. 行式 128 bit UB 把 GEMV 映射为 1×1 卷积，不增加第二套数据通路。
7. Python/C 双实现与逐 token、逐字节 golden 对拍，两个应用共用同一加速器。
8. 私有存储加 uncached mailbox 实现 AMP 聚合；精确定时器中断支撑 RTOS 抢占。

1.6 设计流程

在 ARES 仓库内以 Chisel 生成 Mundus RTL，集成可综合存储、外设和加速器；编译 ELF 并拆分为 IROM/DRAM HEX。通过 Verilator、模块测试和板级仿真核对功能，再使用 PDS 执行综合、布局布线、时序检查与位流生成。每个配置分别保存镜像、日志和验收结果。

第二部分　系统组成及功能说明

2.1　整体介绍

ARES 以 Mundus 处理器为核心，前端经 IROM 请求/响应接口连接 ICache 或直通适配器，后端经含 DCache 的 LSU 和 PERIP 接口访问数据存储及外设。处理器内部由前端预测和取指、译码、发射与寄存器读取、执行、LSU、乘除法及写回单元构成，模块关系见图 2。HZU 根据分支结果协调重定向，ISU 根据数据就绪状态决定旁路与互锁，WB 保持顺序提交。图 1 表示单核 SoC 的连接概览。AMP 为每 hart 例化 `student_top`、私有 IROM/ICache/数据 BRAM 和加速器，另连共享 uncached mailbox。`pipeline_mc_soc_top` 则是共享 L2 的独立实验配置。

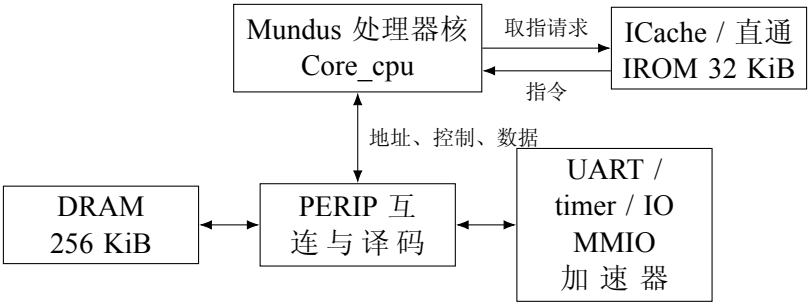


图 1: ARES 单核 SoC 连接概览（DCache 位于核内 LSU）

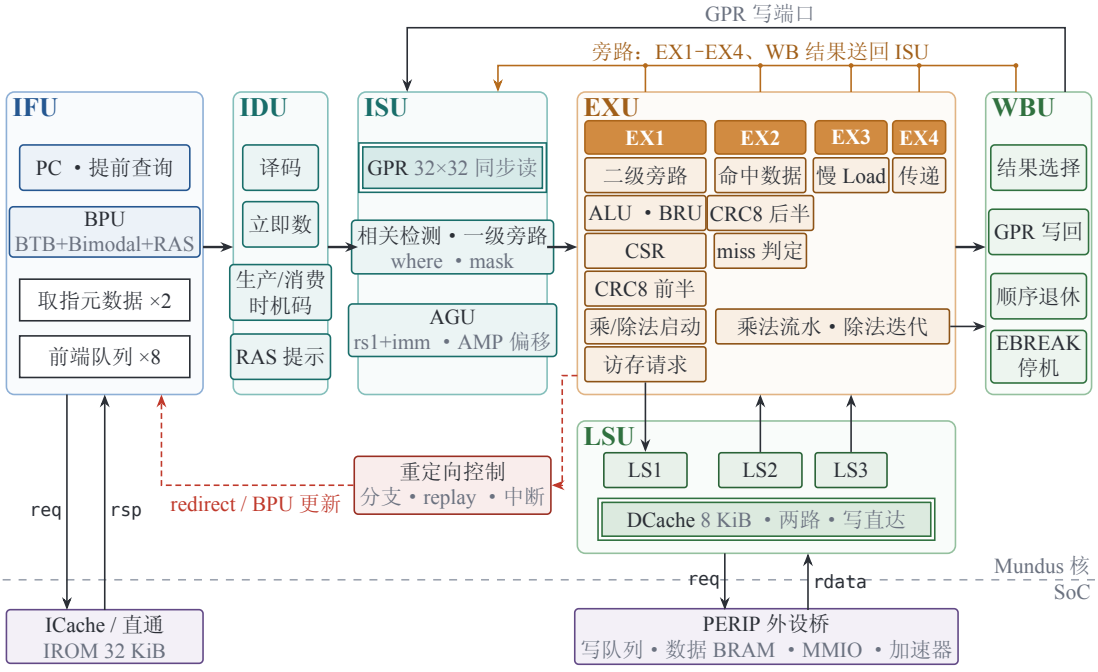
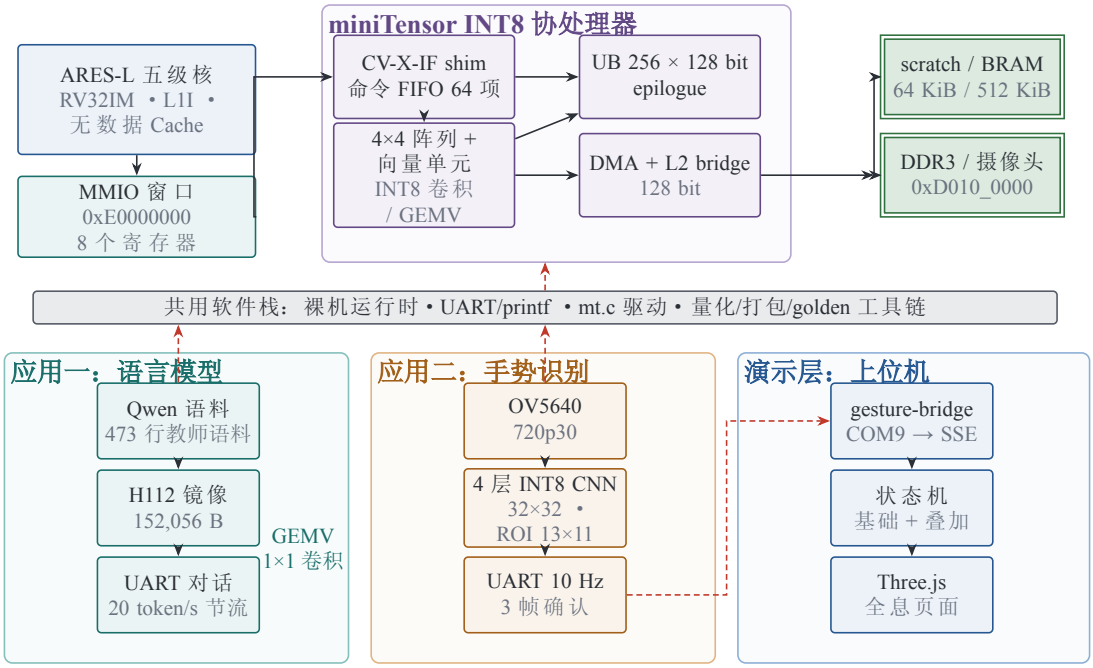


图 2: Mundus 核内模块关系

平台与口径

应用与加速器部分统一以 ARES-L 五级应用基线为口径：CPU 是 ARES-L 五级顺序核，数据按 uncached 视图访问，miniTensor 通过 CV-X-IF 兼容 shim 接入。本部分提到的 CPU 都指这一基线。图 3 给出总体关系：CPU 经 MMIO/CV-X-IF 发命令，DMA 在 UB 与 scratch/BRAM 间搬数，两个应用共用一套软件栈。



同一套 miniTensor、驱动与量化工具链；应用差异只在模型与后处理

图 3: 应用与加速器总体架构（同一 CPU、同一 miniTensor、两条应用路径）

2.2 硬件系统介绍

2.2.1 硬件整体介绍

核心采用 Chisel [1] 描述，并生成 SystemVerilog 与手写乘除法 RTL、BRAM 和外设包装集成。主线目标为盘古 200Pro+ / PG2L200H-6IFBB676，使用 PDS；前身 Mundus 在 Xilinx Kintex-7 / Vivado 2023.2 上的验证资料见附录 B。板级 PLL 由 27 MHz 产生 100 MHz CPU 与 50 MHz 外设时钟；100 MHz 为 PLL 配置值和仿真时钟，布线后时序结论见第 3.3.5 节。AMP 将各 hart 的 CPU、外设桥和 UART 连接到同一 100 MHz 域，板级 50 MHz 输出不参与 AMP 数据通路。

2.2.2 机械设计介绍

作品依托现有 FPGA 试验平台实现，不包含独立机壳、传动机构或机械装配设计。

2.2.3 电路各模块介绍

SoC 集成方式

Mundus 经手写 myCPU 包装接入 student_top，其中 Core_cpu 是实例名。单核板级顶层为 pipeline_board_top，AMP 板级顶层为 pipeline_amp_board_top；RTOS 与 MSH 另有独立板级顶层和镜像。

核心接口

以下使用 Chisel Bundle 信号名，方向均相对 Mundus 核；myCPU 包装将点号展开为下划线。IROM 的请求与响应分别握手；PERIP 请求在 valid && ready 时接受，读结果由 rvalid 标记，读响应没有独立 ready。wen 表示读写属性。

表 2: Mundus 核接口

信号	方向	位宽	说明
irom.req_valid / req_addr	输出	1 / 32	取指请求及字节地址
irom.req_ready	输入	1	请求接收能力
irom.rsp_valid / rsp_data	输入	1 / 32	有效指令响应
irom.rsp_ready / flush	输出	各 1	响应接收、丢弃旧路径事务
perip.valid / addr	输出	1 / 32	数据请求及字节地址
perip.wen / mask / wdata	输出	1 / 2 / 32	写属性、宽度编码、写数据
perip.ready / rvalid / idle	输入	各 1	接受请求、读返回有效、桥空闲
perip.rdata	输入	32	外部读数据
timer_irq / data_offset	输入	1 / 32	定时器中断、私有数据地址偏移
coh.inv_valid / inv_addr	输入	1 / 32	实验版一致性失效及地址
coh.epoch	输入	8	实验版一致性版本号
retire.valid / rd_wen	输出	各 1	退休有效、GPR 写使能
retire.is_ebreak	输出	1	EBREAK 退休标记
retire.pc / pc_next / wdata	输出	各 32	退休 PC、真实下一 PC、写回数据
retire.rd	输出	5	写回寄存器编号

perip.mask 的 00/01/10 分别表示 byte/half/word，11 表示无访存；不等同于逐字节写选通。单核与 AMP 将一致性输入接零；共享 L2 实验版连接失效与 epoch。复位入口为 0x80000000。

主要参数

核参数以 rtl/pipeline/gen/configs/scsrc/autoconf.scala 为准，手写包装参数见 rtl/pipeline/include/pipeline_autoconf.vh，ICache 几何取决于实际顶层例

化。

表 3: 默认参数及顶层覆盖

项目	参数及适用配置
BPU / 前端	BTB 索引 4 bit，共 16 项；Bimodal 2 bit；RAS 深度 8；前端队列 8 项；在途元数据 2 项
单核 ICache	`student_top` 默认 32 组 × 4 路 × 32 B = 4 KiB
AMP ICache	`pipeline_amp_top` 默认 64 组 × 4 路 × 64 B = 16 KiB/hart；每 hart 内部 IROM 为 32 KiB
AMP 板级 ICache	`pipeline_amp_board_top` 覆盖为 128 组 × 4 路 × 64 B = 32 KiB/hart
DCache	8 KiB，1024 组 × 2 路 × 4 B；每次只回填一个字，写直达、写 miss 不分配
外设桥写队列	深度 4；FIFO 排出，Load 按字节转发最新的较老 Store
单核 / AMP 数据 BRAM	单核默认 256 KiB；AMP 默认每 hart 32 KiB，`DRAM_AW=13`、`DATA_STRIDE=0x8000`
乘除法时序	乘法流水延迟 4；除法控制器固定延迟参数 35，底层为 16 次 radix-4 迭代
可选能力	RAS 开启；loop predictor、RV32E、可选 Z 位操作以及 `mscratch/misa` 关闭

AMP 以 `EXTERNAL_IROM=0`、`ICACHE_WAYS=4` 为每 hart 例化私有 IROM 和四路 ICache。

SoC 地址空间

下表为 `student_top/perip_bridge` 单核与 AMP 路径的地址映射，其中 `DRAM` 表示片上数据 BRAM。

表 4: 单核与 AMP 的地址映射

区域	地址或地址范围	属性	说明
IROM	`0x80000000--0x80007FFF`	只读	32 KiB，32 bit 字；同步 BRAM，经 Cache/直通访问
数据窗口	`0x80100000--0x8013FFFF`	读写	单核 256 KiB，DCache 可缓存窗口
SW	`0x80200000 / 0x80200004`	只读	开关低/高 32 bit
KEY	`0x80200010`	只读	低 8 bit 有效
SEG / LED	`0x80200020 / 0x80200040`	读写	数码管 / LED 寄存器
毫秒计数	`0x80200050`	读写	读毫秒数；写 `0x80000000` 启动、`0xFFFFFFFF` 停止
中断 pending	`0x80200054`	读写	bit0 为 pending，写 bit0=1 确认
UART TX	`0x80200060`	只写	写低 8 bit 发送
UART 状态	`0x80200064`	只读	bit1=TX ready，bit0=RX valid
UART RX	`0x80200068`	只读	读取并消费收到的字节

表 5: CoreMark 兼容窗口、加速器与 AMP mailbox

区域	地址或地址范围	属性	说明
CPU 周期数	`0xA0000050 / 0xA0000054`	只读	64 bit 周期计数器的低/高 32 bit
UART TX 别名	`0xA00003F8`	只写	对应原生 TX 寄存器
UART 状态别名	`0xA00003FC`	只读	bit1=TX busy，bit0=RX valid；与原生状态的 bit1 极性相反
Hart 信息	`0xA0000FF0 / 0xA0000FF4`	只读	hart ID / 配置的 hart 数
矩阵引擎	`0xA0001000--0xA0001FFF`	读写	`mtx9_engine` MMIO 页
状态机引擎	`0xA0002000--0xA0002FFF`	读写	`state9_engine` MMIO 页
链表引擎	`0xA0003000--0xA0003FFF`	读写	`list9_engine` MMIO 页
共享 mailbox	`0xA000F000--0xA000F0FF`	读写	仅 AMP 顶层连接；uncached，4 槽 × 16 字

AMP 默认每 hart 配置 32 KiB 私有数据 BRAM。软件链接窗口为 `0x80100000--0x80107FFF`；AGU 对数据窗口地址加 `hart_id * 0x8000`，本地 BRAM 以低地址位索引。IROM 和 MMIO 保持原地址。AMP 板级仅引出 hart0 UART TX，RX 接空闲；独立 MSH 板级顶层另接 B24 RX、C24 TX。

指令集支持

指令编码与语义遵循 RISC-V 非特权规范 [2]，CSR 与 trap 行为遵循机器模式特权规范 [3]，实现范围如表 6 所示。

表 6: 指令集支持情况

类别	支持指令
RV32I 算术/逻辑	`add`, `sub`, `addi`, `and`, `or`, `xor`, `andi`, `ori`, `xori`, `lui`, `auipc`
RV32I 移位/比较	`sll`, `srl`, `sra`, `slli`, `srli`, `srai`, `slt`, `sltu`, `slti`, `sltiu`
RV32I 访存	`lb`, `lh`, `lw`, `lbu`, `lhu`, `sb`, `sh`, `sw`
RV32I 控制流	`beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu`, `jal`, `jalr`
RV32M	`mul`, `mulh`, `mulhsu`, `mulhu`, `div`, `divu`, `rem`, `remu`
Zicsr	`csrrw`, `csrrs`, `csrrc`, `csrrwi`, `csrrsi`, `csrrci`
环境与特权	`ecall`, `mret`、机器定时器中断；`ebreak` 退休后停机
屏障	`fence` 等待 LSU 和外设桥空闲；`fence.i` 暂复用屏障，无 ICache 失效功能
自定义扩展	custom-0 `CRC8`，两阶段完成

总体结构

Mundus 采用顺序、单发射、单提交结构，每周期最多接收并提交一条指令。流水线按取指请求、取指响应、ID、ISS、EX1、EX2、EX3、EX4 和 WB 划分，共九级；表 7 列出各级职责，图 4 给出各级与 LSU、长操作通路的对应关系。IF1/IF2 分别表示取指请求和响应，二者通过握手和队列衔接。

表 7: 取指功能及核内流水阶段

阶段或功能	职责
取指请求（IF1）	PC 选择、BPU 提前查询、IROM/ICache 请求；仅握手后推进正常取指 PC
取指响应（IF2）	两项元数据队列与响应配对，再经八项前端队列送往 ID；Cache miss/反压时延迟可变
ID	译码、立即数、控制信号和源寄存器地址；提前发起同步 GPR 读取
ISS	同步 GPR 数据对齐、相关检查与旁路、提前 AGU；向 DCache 发起查表
EX1	ALU、BRU、CSR、CRC8 第一阶段、启动乘除法；向 LSU 的 LS1 提交访存地址和 Store 数据
EX2	DCache 命中结果及 CRC8 第二阶段；按寄存的命中结果修正 Load 就绪时机；消费 HZU 寄存后的重定向
EX3	选择外部返回的慢路径 Load 数据，已命中结果继续传递
EX4	传递执行结果、寄存器写回和退休元数据至 WB
WB	选择普通/乘除法结果，GPR 写回与顺序退休；EBREAK 触发停机

BRU 在 EX1 计算真实控制流，HZU 寄存一拍后发出重定向，清除错误路径的前端、ID/ISS 和 ISS/EX1 内容，保留更老的后端指令。各级流水寄存器及 LSU 的 LS2/LS3 只对 valid 做同步复位，payload 由 valid 屏蔽而不复位；因此任意宽度的复位脉冲都会清空在途指令，复位释放后的第一条退休指令必为复位入口。LSU 请求未被桥接受时冻结相关流水级；数据相关在 ISS 互锁并向 EX1 注入气泡；长操作未完成时保持顺序提交。Load 在 EX1 按 DCache 命中预测生产时机，EX2 以寄存的命中结果修正：紧邻消费者命中时停顿一拍、miss 时停顿两拍，不撤销已发射指令；EXU 另保留 `load_miss_replay` 作为保护路径（第 2.2.3 节）。

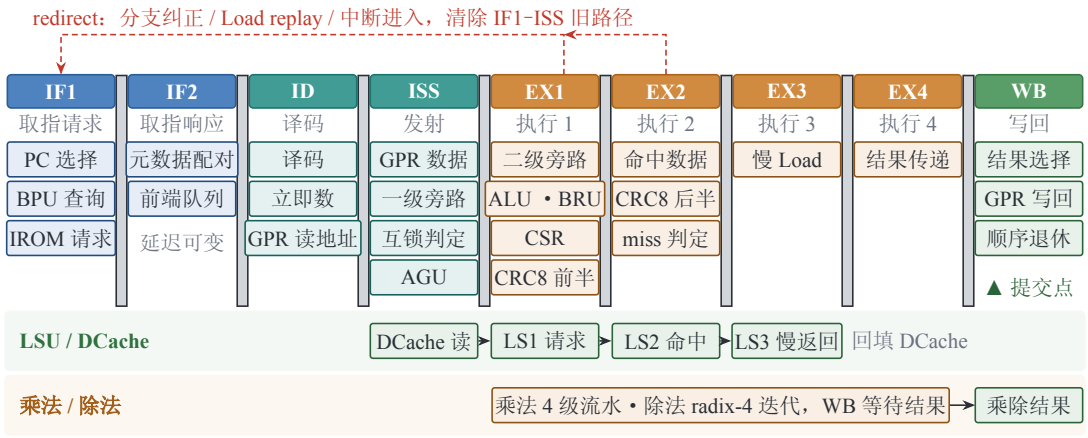


图 4: Mundus 九级流水线：各级职责、LSU 与长操作通路

RTL 前端设计

前端由 PC 选择逻辑、BPU、ICache/直通接口、两项取指元数据队列、八项前端队列和 IF/ID 输出通路组成。图 5 给出前端结构，ICache 及其队列配合详见下文。BPU 采用常见的动态预测组合 [4]，以 Bimodal 预测器判断跳转方向，通过 BTB 保存历史跳转目标，并可选配返回地址栈 RAS 预测函数返回地址。PC 用于驱动 IROM 取指并生成对应的请求元数据；IROM 返回指令后，与 FetchMeta 中保存的 PC 及预测信息对齐，再经前端队列送入 IDU。

RAS 对函数调用与返回较频繁的程序具有较明显的预测收益，**但对于此类控制流较少的程序，准确率改善可能较为有限；与此同时，RAS 及其更新、选择逻辑也会增加一定的硬件资源和布局布线负担**。因此, 本设计将 RAS 实现为由 `CONFIG_BRANCH_RAS` 控制的**可选配置**。启用该配置时，生成的可综合 RTL 包含 RAS、返回指令标记以及相应的更新和目标选择逻辑；关闭该配置时，上述模块、接口和数据通路均不参与硬件例化，BPU 仅使用 Bimodal 与 BTB 完成分支预测。

ARES 核参数集中在 `gen/configs/scsrc/autoconf.scala`，由 `make pipeline-core-gen` 生成对应 RTL；开关 `CONFIG_BRANCH_RAS` 即可得到两种预测器配置。

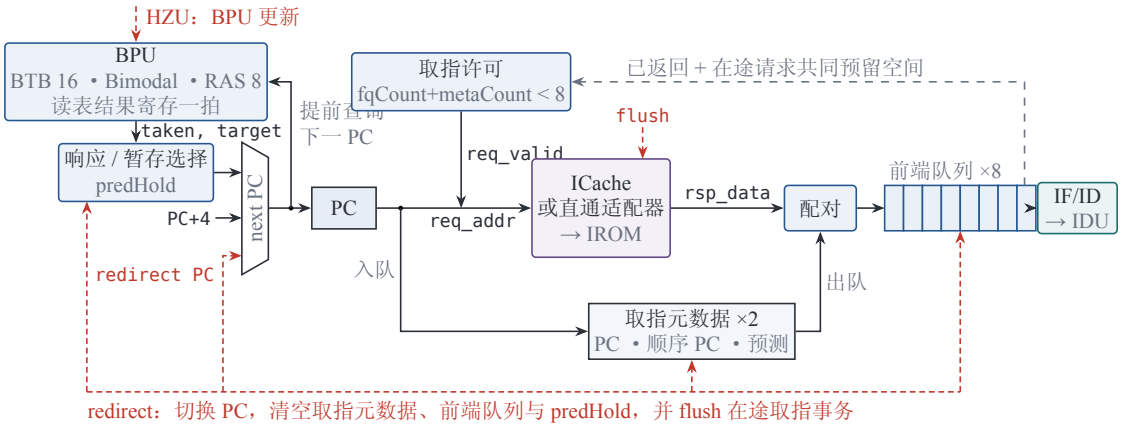


图 5: 前端取指结构：提前查询、请求许可与两级队列

分支预测单元 BPU

Bimodal 方向预测

BPU 使用 Bimodal 预测器 [5] 学习分支的历史跳转倾向。每个已记录的控制流指令对应一个 2 bit 饱和计数器 counter，计数器的最高位给出方向预测：SNTK 和 WNTK 预测不跳转，WTK 和 STK 预测跳转。Bimodal 的方向状态在物理实现上与 BTB 表项一一对应，不再单独设置 PHT，从而复用同一组索引和读出寄存器。

表 8: Bimodal 2 bit 饱和计数器状态

状态	编码	预测倾向
SNTK	2'b00	强不跳转
WNTK	2'b01	弱不跳转
WTK	2'b10	弱跳转
STK	2'b11	强跳转

EX1 得到真实控制流结果后，经 HZU 寄存一拍，将实际方向反馈给预测器。对应索引项已经有效时，计数器根据实际结果向 STK 或 SNTK 饱和更新；空表项首次记录一次实际跳转时，计数器从 WTK 开始。复位时计数器初始化为 WNTK，使尚未完成训练的表项倾向于预测不跳转。

返回地址栈 RAS

间接跳转指令 `jalr` 的目标地址由寄存器操作数决定，仅依赖 BTB 中上一次保存的目标难以稳定预测函数返回。为此，BPU 增加可选配置返回地址栈 RAS [6]，专门记录尚未返回的函数调用地址。默认 RAS_WIDTH=3，栈深度为 $2^3 = 8$。RAS **采用移位式栈结构**：执行 push 时，原有表项依次向栈底移动，并将新的返回地址写入 `stack[0]`；执行 pop 时，后续表项依次向栈顶移动。因此，`stack[0]` 始终保存当前栈顶地址，预测端可以直接读取该地址，无需通过栈指针对多个表项进行动态选择。

IDU 根据 `jal/jalr` 的 `rd` 和 `rs1` 字段生成 RAS 更新提示。按照 RISC-V 的约定，`x1` 和 `x5` 均可作为链接寄存器：指令通过链接寄存器保存 PC+4 时产生 ras_push，将新的返回地址压入 RAS；`jalr` 使用链接寄存器作为跳转源、但不通过同一寄存器保存新返回地址时产生 ras_pop。若 `jalr` 分别使用两个不同的链接寄存器作为源和目的，则同时产生 ras_pop 和 ras_push，实现协程切换所需的先弹出旧返回地址、再压入新返回地址。

BTB 表项使用 is_return 标记已识别的返回指令。预测端在 BTB 命中、is_return 有效且 RAS 非空时，将该指令直接预测为跳转，并使用 RAS 栈顶覆盖 BTB 中保存的历史目标；若 RAS 为空，则退回普通 Bimodal 与 BTB 的预测结果。于

是返回预测命中条件及最终选择为

$$\begin{aligned} \text{return_hit} &= \text{pred_is_return} \wedge \text{ras_top_valid}, \\ \text{pred_taken} &= \text{return_hit} \vee \text{bimodal_taken}, \\ \text{pred_target} &= \begin{cases} \text{ras_top}, & \text{return_hit}, \\ \text{btb_target}, & \text{otherwise}. \end{cases} \end{aligned} \tag{1}$$

BTB 表项与命中更新

BTB 采用直接映射结构，默认配置 BTB_WIDTH=4，共有 $2^4 = 16$ 个表项。每个表项包含 valid、tag、历史跳转目标 target、Bimodal 的 2 bit counter 以及返回指令标志 is_return，相关阵列被约束为 Distributed RAM。取指 PC 按 4 字节对齐，PC[5:2] 用作索引，PC[31:6] 用作 tag；索引选出候选表项后，再由 valid 和 tag 比较确认其是否属于当前 PC，基本结构如图 6 所示。

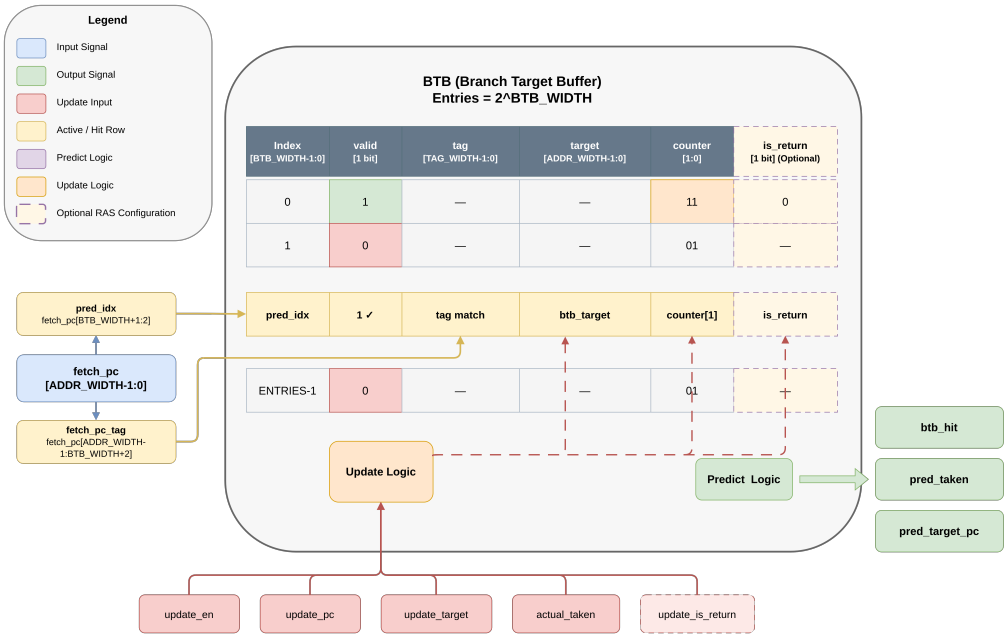


图 6: Bimodal、BTB 与 RAS 原理图（默认 16 项 BTB、8 项 RAS）

预测时，fetch_pc 同时产生索引和 tag。只有候选表项有效且 tag 相等时才命中；普通控制流指令在命中后以计数器最高位判断方向，并使用表项 target 作为跳转目标：

$$\begin{aligned} \text{pred_hit} &= \text{valid_r} \wedge (\text{tag_r} = \text{pred_tag_r}), \\ \text{bimodal_taken} &= \text{pred_hit} \wedge \text{counter_r}[1], \\ \text{pred_is_return} &= \text{pred_hit} \wedge \text{is_return_r}. \end{aligned} \tag{2}$$

未命中或最终预测不跳转时，IFU 选择 PC+4。EX1 的控制流更新经 HZU 寄存后，BTB 写入 valid、tag、target 和 is_return；同时按实际方向训练对应的 Bimodal 计数器。复位时所有 valid 清零、counter 初始化为 WNTK；tag、target 和 is_return 不复位，其无效内容由 valid 屏蔽。控制冒险通过比较预测下一 PC 与真实下一 PC 判断，两者不一致时由 HZU 发出 redirect。

一拍预测与提前查询

若在一个周期内依次完成预测表读取、tag 命中比较、counter 方向判断以及 target 与 PC+4 的选择，整条组合逻辑会直接连接到 PC 寄存器输入，路径较长。**为缩短该路径**，将预测表读出结果寄存：第 N 拍输入要预测的 PC 和请求使能，时钟沿锁存请求 PC 及表项的 valid、tag、target、counter 和 is_return，第 $N+1$ 拍输出预测响应，使预测表阵列的读取不再位于同拍的 PC 更新路径中。

一拍接口也意味着，若直接查询当前 PC，必须到下一拍才能确定预测取指地址，连续取指会产生气泡。IFU 因而采用提前查询。设本拍 PC 为 P，BPU 已返回 P 的预测，IFU 组合得到候选下一地址 Q：

$$Q = \begin{cases} \text{pred_target}(P), & \text{pred_taken}(P), \\ P + 4, & \text{otherwise.} \end{cases} \tag{3}$$

当本拍取指请求被接受时，PC 在时钟沿更新为 Q，同时 BPU 查询端直接接收 Q，而 IROM 仍接收当前地址 P。下一拍 BPU 对 Q 的预测与已经更新为 Q 的 PC 对齐，可以继续计算再下一地址。这样一拍预测延迟被流水隐藏，稳态仍可每周期接受一条取指请求。

完成一拍流水化后，预测到 PC 寄存器的组合路径仍然较长，其主要经过：BTB/Bimodal 读出寄存器 $\rightarrow$ valid/tag 命中比较、counter 方向判断与返回类型判断 $\rightarrow$ RAS 栈顶目标覆盖 $\rightarrow$ 当前响应与暂存预测选择 $\rightarrow$ 预测目标与 PC $+ 4$ 选择 $\rightarrow$ PC 寄存器输入。该路径虽然不再包含预测表阵列的读取，但仍包含 tag 比较、RAS 选择、多级选择和 32 bit 加法，仍是前端综合与布局布线时需要重点观察的关键路径。

预测保持与重定向清理

预测保持机制用于解耦 BPU 响应与 IFU 取指请求：BPU 的响应只在对应周期有效，但 IFU 可能因后端停顿而暂时无法发出请求。若不保存该响应，恢复取指时会丢失已经得到的预测方向和目标，并直接进行顺序地址选择，**很有可能会**

导致预测错误，使 IPC 降低。

为了解决上面提到的问题，添加了 predHoldValid、predHoldTaken 和 predHoldTarget 寄存器来保存这一次预测。后续请求恢复时优先使用暂存结果，并在请求被接受后清除暂存有效位。固定的一拍响应保证了同一时刻最多只有一个需要暂存的预测。在重定向的时候，redirect 信号用于防止错误路径的预测在流水线冲刷后再次生效。当redirect 拉高时，IFU 在当拍禁止新取指并清除预测暂存，同时向 BPU 查询重定向目标。下一拍返回的是新目标的预测；元数据和前端队列也同时清空，旧路径指令不会再次被消费。

取指单元 IFU 与 IROM 握手

底层 `IROM_BMEM32` 是 8192 × 32 bit 的同步存储包装，直通适配器读延迟为一拍；核心所见响应延迟由所选 ICache/适配器决定。IFU 在 `req_valid && req_ready` 时将 PC、顺序 PC 和预测信息入队 `metaVec`，在响应握手时用队首元数据配对指令，并推进 `metaCount`；最多两项在途请求。八项前端队列保存已返回指令，发出请求前按队列占用和在途数量预留空间，避免后端停顿造成响应丢失。Cache 负责在 `rsp_ready` 为低时保持响应。

表 9: 取指握手、背压与重定向规则

事件	PC、元数据和指令的处理
请求握手	推进 PC/BPU，并将本次请求的 PC 与预测信息入队
响应握手	队首元数据与指令配对，送入前端队列或直接交给下游
Cache miss / 反压	未握手请求不推进 PC；已接受请求继续占用预留槽
后端停顿	已返回指令留在前端队列；空间用尽后停止发新请求
Redirect	重置 PC，清元数据与前端队列、旧预测暂存；`irom.flush` 清理取指事务状态

单核默认四路 ICache 命中延迟为两拍，流水化命中路径可每拍接受一条请求；refill 和重定向会增加等待。

指令缓存 ICache

接入位置与设计目标

ICache 位于 IFU 与只读 IROM 之间，提供每次一条 32 bit 指令的请求/响应接口。其目标是将前端的取指节奏与底层同步存储器解耦，并利用流水化命中路径维持单发射所需的每拍一条取指吞吐。连接关系为 IFU → `pipeline_icache` → `pipeline_irom_passthrough` → `IROM_BMEM32`。ICache 以整行缓存指令，回填适配

器将一行拆为连续的 32 bit IROM 读取，内部结构见图 7。

`USE_ICACHE=1` 选择缓存路径，`USE_ICACHE=0` 仅保留一拍直通适配器，两者向 IFU 提供相同的握手信号。`ICACHE_PIPE_HITS=1` 允许命中流水重叠，置 0 则保留串行请求节奏，用于对照取指吞吐的影响。AMP 在每个 `student_top` 内例化同一主线四路缓存；各 hart 拥有私有缓存和私有 32 KiB IROM，执行相同的只读代码镜像。

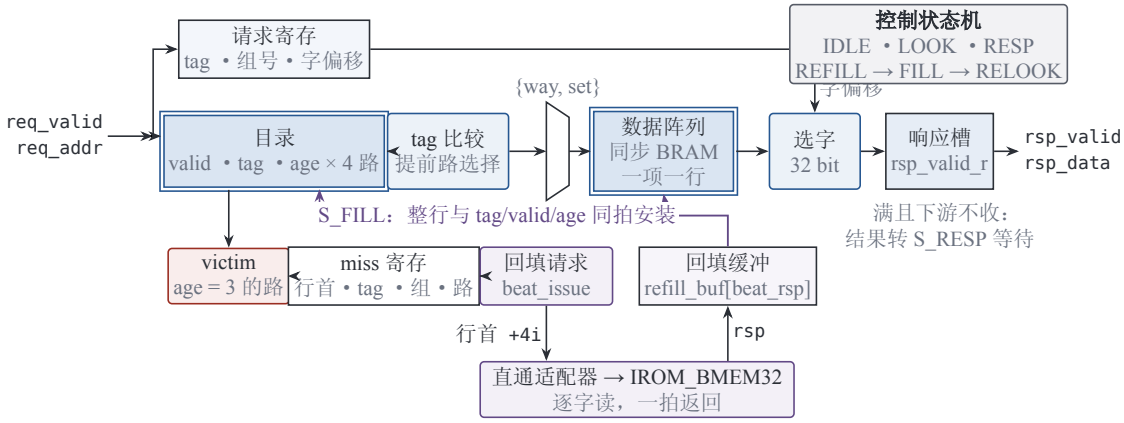


图 7: 四路 ICache 内部结构

容量、地址划分与存储组织

缓存容量满足 $C = S \times W \times B$，其中 S 为组数，W 为路数，B 为行字节数。三类配置的地址划分见表 10；其中 AMP 的 `ICACHE_LINES` 表示总行数，例化主线缓存时除以 4 得到组数。

表 10: 四路 ICache 配置与地址划分

配置	容量	组数	tag	index	行内偏移
单核默认	4 KiB	32	`PC[31:10]`	`PC[9:5]`	`PC[4:0]`
AMP 默认/hart	16 KiB	64	`PC[31:12]`	`PC[11:6]`	`PC[5:0]`
AMP 板级/hart	32 KiB	128	`PC[31:13]`	`PC[12:6]`	`PC[5:0]`

指令按 4 字节对齐，行内偏移最低两位不参与选字，其余位选择该行中的第几条指令。32 B 行包含 8 条指令，64 B 行包含 16 条指令；返回端始终只输出其中一条，不进行跨行拼接，也不支持 16 bit 压缩指令的半字取指。

目录以展平数组保存每组各路的 valid、tag 和 age，数据阵列则以 `{way,set}` 为地址，一项存放完整缓存行。单核默认数据项宽度为 256 bit，AMP 配置为 512 bit。目录在组合逻辑中比较请求 tag，提前选择数据阵列读地址；同步 BRAM 只读命中路的一整行，再根据保存的字偏移选择 32 bit 指令。这避免了为了并行读

出四路数据而复制整套数据阵列，代价是 tag 比较与路选择位于 BRAM 读地址之前。

命中流水与响应保持

设周期 T 中请求 P 握手：时钟沿保存其 tag、组号和字偏移，并完成对应数据行的同步读取；接下来的 `S_LOOK` 周期确认命中，将选出的指令写入响应槽，随后对 IFU 置 `rsp_valid`。按该接口的周期划分，命中延迟为两拍（图 8(a)）。

缓存以 `out_free = !rsp_valid_r || rsp_ready` 判断响应槽是否可写。响应槽已满且下游不接收时，原响应保持不变，当前查找结果转入 `S_RESP` 等待；空间释放后再写入该结果。因此，已返回的响应不会因下一次查找而被覆盖。`PIPE_HITS=1` 时，`S_LOOK` 命中且响应槽可写的同拍可以接受下一个请求，将当前结果写入响应槽，同时为下一请求读取数据行，稳态达到每拍接受一条请求、每拍返回一条指令。请求到响应的命中延迟为两拍，连续命中吞吐为一条/拍。流水最多同时容纳一个响应槽和一个查找请求，miss 期间暂停接受新请求。

替换策略与逐字回填

每组四路的 age 保持为 $0, 1, 2, 3$ 的一个排列，0 表示最近访问，3 表示最久未访问。命中路更新为 0，原来比它更新的各路 age 加 1；回填安装时也按此规则将被填入路设为最近使用。未命中时选择 age 为 3 的路作为 victim，并锁存行首地址、tag、组号和路号。该 age 排序实现四路 LRU 替换。

回填阶段分别维护 `beat_issue` 和 `beat_rsp`：请求握手才推进发出计数，响应握手才将数据写入对应 `refill_buf` 项并推进接收计数。第 i 个请求的字节地址为 line_base $+ 4i$，单核默认发出 8 个字请求，AMP 的 64 B 行发出 16 个。最后一个响应到达后，缓冲中的字按地址递增顺序组成整行，再写入数据阵列和目录。缓存等待整行完成后重新查找原请求，每次处理一个 miss；以 32 B 行为例，从请求握手到响应有效共 14 拍（图 8(c)）。

表 11: ICache 状态与转移条件

状态	工作及后续状态
`S_IDLE`	等待请求握手，保存地址并进入 `S_LOOK`
`S_LOOK`	命中且响应槽可写时提交响应；若同拍接受新请求则继续查找，否则回到空闲。响应槽被占用则转 `S_RESP`；miss 则锁存 victim 并进入回填
`S_REFILL`	逐字发出请求并接收响应，收齐一行后转 `S_FILL`
`S_FILL`	安装数据行，写 tag/valid 并更新 age，随后转 `S_RELOOK`
`S_RELOOK`	从新安装的数据行发起同步读取，再转 `S_LOOK`
`S_RESP`	保持待提交结果，响应槽可写后提交；可同时接收下一请求

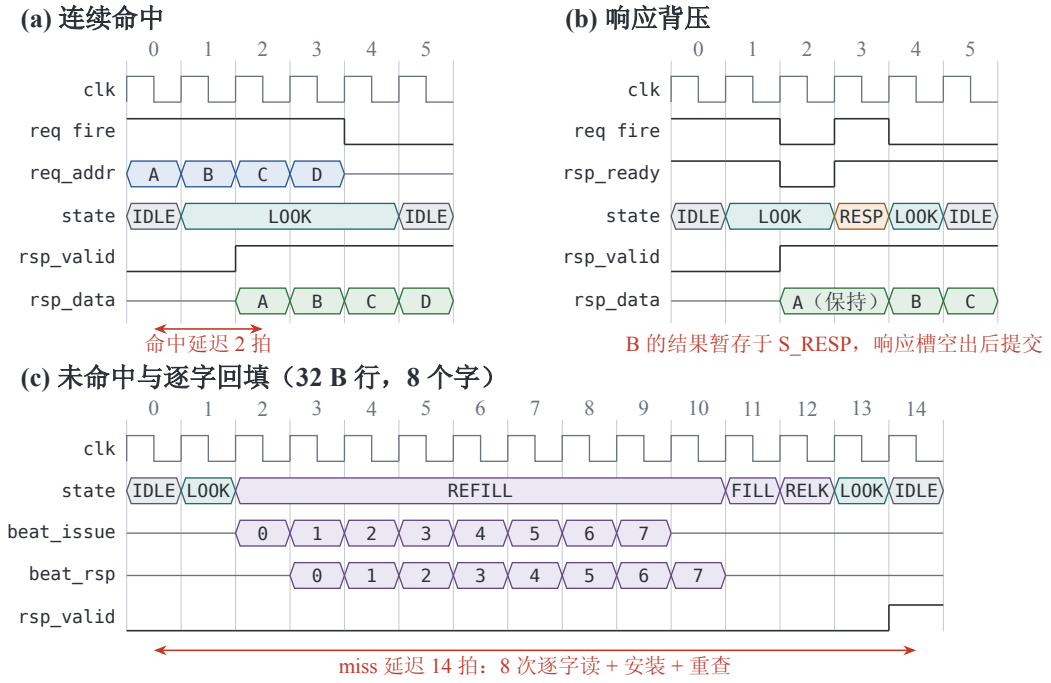


图 8: ICache 连续命中、响应背压与未命中回填时序（32 B 行，PIPE_HITS=1）

IFU 元数据与前端队列的配合

ICache 只返回指令字，不回送 PC、请求编号或预测信息。因此 IFU 要求响应按请求接受顺序返回，并用两项元数据队列 `metaVec` 保存每个在途请求的 PC、顺序 PC、预测方向与预测下一 PC。请求握手将元数据压入队尾，响应握手将队首与 `rsp_data` 配对。压入和弹出同拍发生时复用腾出的槽位，使前端不必等队列完全清空才继续取指。

已配对的指令进入深度为 8 的 fall-through 前端队列。队列为空且 ID 可接收时，响应直接旁通给下游；队列已有较老指令或下游暂停时，响应存入队尾，下一次优先输出队首，保持程序顺序。IFU 使用 reservedCount = fqCount + metaCount

预留空间，只有该值小于 8 且元数据队列有容量时才允许发出新的取指请求。判断有意不计入本拍可能出队的收益，使后端 ready 不直接串回 PC/BPU 请求生成路径。八项容量由已返回指令和在途请求共同预留。

重定向、复位与统计

分支纠正、中断进入以及 Load miss 保护路径均通过 IFU 的 redirect 清理旧路径：禁止本拍新取指，清空元数据和前端队列，切换 PC，并向 ICache 及其直通回填适配器同时发出 `flush`。缓存清理事务状态和响应有效位，停止接收当前回填，并保留目录中的 tag/valid。直通适配器同时清除 pending，丢弃旧响应。

安装与 flush/复位同拍的一致性。数据阵列写使能与目录安装共用 `fill_we = (state==S_FILL) && !rst`。安装当拍若发生 flush，整行数据已全部到达且 IROM 只读，因此照常写入数据并安装 tag/valid/age，只丢弃在途请求和响应；若当拍复位，则数据与目录均不写入。早期实现的数据写只由 `S_FILL` 产生，目录安装却位于 rst/flush 之后的分支内，flush 同拍会保留旧 tag 却写入新行数据。`pipeline-icache-test` 覆盖这一情形：先填入同组的 `0x80000000/0x80000400/0x80000800/0x80000C00`，对 `0x80001000` 的回填在 `S_FILL` 当拍分别施加 flush 和复位，再重取五行核对数据；随后的随机阶段混合两侧背压、随机 flush 以及针对安装周期的 flush/复位。修复前该测试在首个定向用例即读到错误指令，修复后全部通过。

普通 `rst` 清状态机、响应槽和计数器，将每组 age 置为路号，保留 valid/tag/data。该复位方式要求 IROM 镜像保持不变；有效位的上电初始化待验证。`FENCE.I` 复用访存屏障，ICache 失效功能尚未实现。

模块提供请求、miss、安装完成及响应提交四个 64 bit 计数器，`student_top` 在 EBREAK 退休时触发统计输出。其中 `hit_count` 是响应提交计数，包含 miss 回填后重查成功的响应。性能记录分别列出请求、miss、完成回填和响应提交数，并注明 Cache 配置与镜像。

本节对应 `core/vsrc/icache/pipeline_icache.v`、`core/scsrc/ifu/IFU.scala` 和 `soc/jyd/pipeline_irom_passthrough.v`，路径均相对 `rtl/pipeline/`。集成验证入口包括 smoke、ISA、CoreMark 及 Cache 开关对照测试。

译码单元 IDU

IDU 从 IF/ID 接口接收 32 bit 指令，解析 opcode、rd、rs1、rs2 与 CSR 地址，并按 R、I、S、B、U、J 和 CSR 格式生成立即数。集中式译码表同时生成 ALU 操作数选择与操作码、BRU/MDU/CSR 控制、寄存器写使能、访存方向、访存宽度、

load 符号扩展方式，以及源操作数最晚需求时刻等控制信息。PC、snpc 和分支预测结果不在 IDU 中重新计算，而是随 ID/ISS payload 继续传递，供后续 BRU 校验预测结果。

启用 RAS 后，IDU 还根据 `jal/jalr` 的寄存器字段生成 RAS 更新提示。按照 RISC-V 对链接寄存器 `x1` 和 `x5` 的约定，`jal` 或 `jalr` 写入链接寄存器时置位 ras_push；`jalr` 从链接寄存器读取返回地址，并且目的寄存器不是同一个链接寄存器时置位 ras_pop。若 `jalr` 的源、目的分别使用不同的链接寄存器，则两个信号同时置位，对应协程切换所需的先 pop 后 push。IDU 只产生提示，不在译码阶段直接修改 RAS，以免错误路径指令过早改变预测器状态。

RTL 后端设计

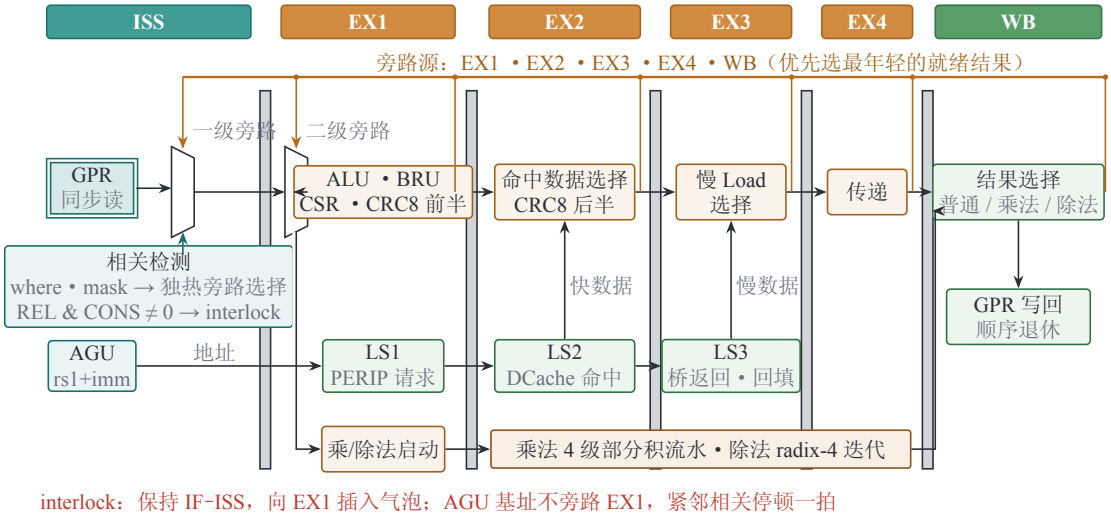


图 9: 后端执行通路与两级旁路

后端由 ISU、EX1–EX4、LSU、乘除法单元和 WBU 组成，数据通路与两级旁路如图 9 所示。

发射与旁路单元 ISU

ISU 负责 RegisterFile 旁路修正、数据相关检查和发射。ISU 的 interlock 信号，为 ISS 流水线阶段自锁信号，依据冒险在哪一流水线阶段产生，生产者能否满足消费者的需求而生成。(旁路及冒险相关内容，在第 2.2.3 小节详细阐述) RegisterFile 的读地址由 ID 提前提供，使同步读数据在指令进入 ISS 时对齐；相关、长操作、桥反压或中断排空造成保持时，读使能关闭，已读数据与寄存的提前比较结果一同保持，避免与 ISS 指令错位。

本设计将访存地址的计算提前到 ISS 阶段，这虽然会导致 AGU 无法旁路 EX1 的数据，产生一个周期的 use-load stall，但是可以减少一个周期的 load-use

stall。根据我们对汇编代码的分析，use-load stall 的发生频率，往往比 load-use stall 要低。

```
src0_irom_data.asm              insn=  2035 load=  355 lu= 226 ul= 106
withMext_pure_irom_data.asm     insn=  1803 load=  258 lu= 140 ul= 119
withoutMext_rv32m_irom_data.asm insn=  1504 load=  232 lu= 119 ul= 101
src1_rv32m_irom_data.asm        insn=  1909 load=  326 lu= 202 ul=  97
init_rv32m_irom_data.asm        insn=  1271 load=  170 lu=  98 ul=  85
init_irom_data.asm              insn=  1271 load=  170 lu=  98 ul=  85
src0_rv32m_irom_data.asm        insn=  2035 load=  355 lu= 228 ul= 106
withMext_rv32m_irom_data.asm    insn=  1803 load=  258 lu= 140 ul= 119
src2_rv32m_irom_data.asm        insn=  1996 load=  326 lu= 194 ul= 103
TOTAL                           insn= 15627 load= 2450 lu=1445 ul= 921
```

图 10: load-use 和 use-load 发生频率对比分析

为了降低“AGU 提前计算”的设计对主频带来的影响，ISU 为 AGU 配备了专用的提前比较与独热码旁路网络。AGU 在 ISS 阶段计算 rs1+imm，对紧邻生产者（EX1）无法直接旁路的情况插入一个周期的 interlock，随后从 EX2 或更后级选择正确基址。

执行单元 EXU

EXU 由 EX1–EX4 四个阶段组成：EX1 完成 ALU、BRU、CSR、CRC8 第一阶段并发出 LSU/乘除法请求；EX2 接收 DCache 命中数据、完成 CRC8 并修正 Load 就绪时机；EX3 接收慢路径 Load；EX4 向 WB 传递结果和退休信息。

ALU 支持 32 位加减、按位逻辑、移位及有符号/无符号比较。BRU 在 EX1 处理六类条件分支和 JAL/JALR，JALR 目标最低位清零；HZU 将结果寄存一拍后协调重定向。

乘除法单元

Pango 文件表使用 `soc/ip/mult.v` 和 `soc/ip/div.v` 的可综合 RTL，手写 `Multiplier/Divider` 包装连接核接口。乘法器以 33 bit 带显式符号的操作数形成四个部分积，再分级相加，固定四拍输出低 64 bit 乘积；WBU 按 MUL/MULH/MULHSU/-MULHU 选择高低部分。除法底层执行 16 次 radix-4 无符号迭代，核侧负责符号与除零结果，控制器固定延迟参数为 35。较新的 DIV/REM 在已有长操作在途时留在 ISS；WB 等待长操作结果以保持顺序提交。

CSR 与异常处理

CSR 单元通过掩码选择目标寄存器，并对 RW、RS、RC 三类命令执行覆盖、置位和清位。立即数 CSR 指令将 zimm 零扩展后复用同一写数据通路。已实现的

CSR 如表 12 所示。

表 12: 机器模式 CSR

CSR	地址	复位值	实现语义
mstatus	0x300	0x00001800	MIE/MPIE 可写，MPP 固定 M 模式
mie	0x304	0	仅 MTIE（bit7）
mtvec	0x305	0	仅 Direct 模式，低两位清零
mepc	0x341	0	ECALL 的 PC 或中断恢复 PC，低两位清零
mcause	0x342	0	ECALL=11；机器定时器中断 =0x80000007
mip	0x344	MTIP 随输入	bit7 反映 timer_irq，CSR 写不清 pending

mscratch（0x340）和 misa（0x301）为可选 CSR，默认生成配置关闭。ECALL 保存当前 PC 和原因 11，MIE 保存到 MPIE 后清零，再转至 mtvec；MRET 恢复 MIE、置 MPIE 并跳至 mepc。机器定时器中断在 MIE、MTIE 与 pending 同时有效时停止新发射，等待后端、乘除法、LSU 和桥排空，通过寄存的空闲状态进入 trap。mepc 使用最后退休指令的真实下一 PC；中断返回原 mepc，ECALL 软件处理后通常将其加 4。该 CSR 与 trap 流程支持 RT-Thread Nano 单核运行。

数据缓存 DCache

缓存范围与单字组织

DCache 位于 LSU 内部，加速 0x80100000--0x8013FFFF 数据窗口的 Load 结果取得。cacheable 判断比较地址高 14 位；UART、计时器、加速器和 mailbox 等 MMIO 均不参与缓存，避免将具有副作用的寄存器读写当作普通内存访问。AMP 在 ISS 的 AGU 后、DCache 查表前对数据窗口地址加 data_offset，使缓存 tag、组号和后续 BRAM 访问使用同一偏移后的地址；IROM 与 MMIO 不经过该重定位。

总容量为 8 KiB，按 1024 sets × 2 ways × 4 B 组织。每项保存一个 32 bit 字，一次 miss 回填一个字。组索引为 addr[11:2]，tag 为 addr[17:12]，addr[1:0] 是字内字节偏移。可缓存窗口的高 14 位固定，目录仅保存其余地址中的 tag。

每路各自例化一份 DCacheData_BRAM 与 DCacheTag_BRAM，分别保存数据和 {valid,tag}；另有一份每组 1 bit 的 DCacheLru_BRAM 保存最近使用路。各阵列均使用独立写口和同步读口，读地址直接来自 ISS，不在地址端串入命中、互锁或全组选择逻辑。两个 tag 和两个数据字并行读出，tag 比较后优先选择编号较小的命中路。命中判定和数据选择使用同一命中向量，避免二者选择不同的路。整体结构见图 11。

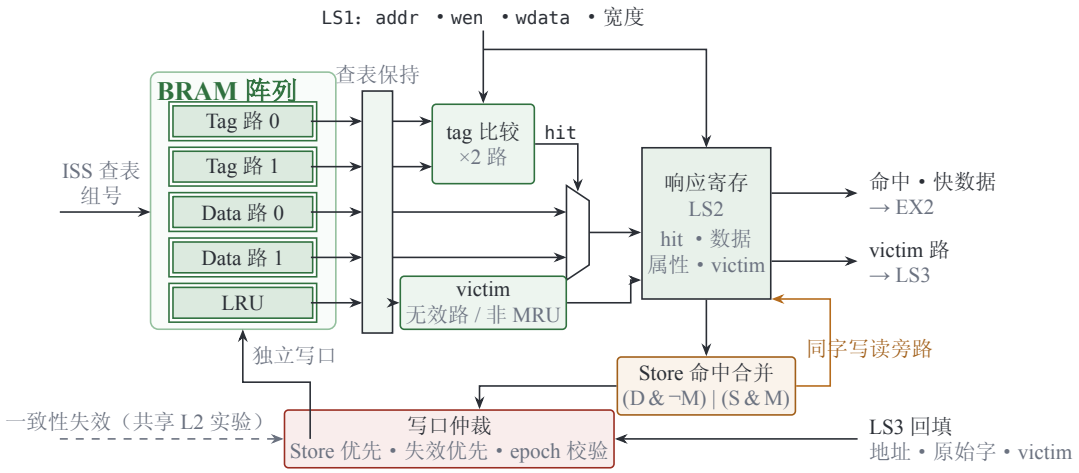


图 11: 两路单字 DCache 内部结构（1024 组 × 2 路 × 4 B）

提前查表、命中返回与请求保持

DCache 的查表与正式接受请求分开进行。ISS 发射访存指令时，`lookup_addr` 已驱动全部 BRAM 的同步读地址；随后 LS1 中的地址及属性与读出结果对齐，只有 LSU 请求实际推进、且地址属于可缓存窗口时，才由 `req_valid` 接受这次缓存访问。命中向量、选中的数据、访问宽度、Store 属性和地址进入响应寄存器，在 LS2 提供给执行流水线。这样把较大的存储阵列读取放在 ISS 到 LS1 之间，LS1 到 LS2 主要承担 tag 比较和两路数据选择。

长操作或桥背压使请求停留在 LS1 时，BRAM 地址仍跟随 ISS。缓存使用 `held_req_valid` 及对应的命中向量、数据和 victim 快照保存已完成的查表结果，直到该请求被接受或不再存在。保持槽保存尚未接受请求的查表结果，响应寄存器保存已接受请求的数据与属性。

替换、初始化与单字回填

每组优先选择无效路；两路均有效时，淘汰最近使用路的另一条路。`lru[s]` 实际保存 MRU 路号，命中访问和成功回填都会更新它；两路情况下一个 bit 即可表达 LRU 顺序。例如，同组的 A、B 均已驻留且最近访问 A，则下一次同组 miss 选择 B 所在路。若命中更新与回填更新争用同一个 LRU 写口，回填优先；丢失一次命中更新只影响替换选择，不改变已保存数据。

victim 在接受请求时确定，并通过 LSU 的 `ls2_cache_way/ls3_cache_way` 随该 Load 传递。外部字返回时直接使用保存的路号安装，避免在回填阶段再读取 LRU 或根据已变化的组状态重新选择路。miss 所需的外部读由 PERIP 发出；LS3 消费返回时提供地址、完整 32 bit 原始字和 victim。LB/LH 等指令虽只写回扩展后的部分数据，缓存仍安装整个原始字，供后续不同宽度访问复用。

复位通过 `tag_init_busy` 逐组清除两路 valid 并初始化 LRU，1024 组需要 1024 次写入；之后再用一拍 `tag_ready` 延迟排除最后一次同步旧读结果。数据 RAM 保留内容，由 valid 控制数据有效性。初始化期间关闭命中和回填安装，Load 通过外部慢路径完成。

写直达、Store 命中更新与旁路

所有 Store 都通过 PERIP 进入后备存储路径，DCache 不持有脏数据。命中 Store 在 LS2 将原字与写数据按字节掩码合并，更新缓存副本；未命中 Store 不分配新项，只完成外部写入，属于 write-through、no-write-allocate 策略。对每个被写字节，合并关系为

$$D_{\text{new}} = (D_{\text{old}} \wedge \neg M) \vee (D_{\text{store}} \wedge M), \tag{4}$$

其中 M 是按地址低位和访问宽度生成的 32 bit 位掩码，写数据先移到目标 byte/half 所在的位置。访存处理限定在一个 32 bit 字内，半字和字访问遵循自然对齐。

Store 更新所有在请求接受时匹配的路，而不只更新选出数据的最低编号路。提前选择 victim 的流水化回填可能使同一字短时出现在两路；同时更新这些副本可以避免后续命中另一路时看到旧值。缓存还比较当前完成 Store 及前一拍写入字与后继请求的字地址，将合并后的完整字旁路给后继访问，减少对 BRAM 同址读写模式的依赖。该旁路处理缓存内的写读相关，外设桥另行处理写队列到 Load 的转发。

回填、写入和失效竞争

同一条数据可能同时受到回填、较年轻 Store、同步查表和实验版一致性失效影响，控制逻辑按表 13 处理。

表 13: DCache 主要竞争及处理规则

竞争情况	处理方式
回填与 ISS 查表同组	寄存被写路号、是否同字及回填数据；下一拍覆盖该路可能不确定的 BRAM 读结果
回填占用了 Store 原命中路	`store_clash` 按路记住同组替换，直到响应完成；抑制向已被替换路写入旧字合并结果
较老 Load 返回、较年轻 Store 已到同字	LS1 或 LS2 出现同字 Store 时，LSU 放弃较老 Load 的缓存安装，避免重新制造旧副本
Store 命中更新与回填同拍	Store 数据写优先，回填放弃；后备存储仍保存完整结果
一致性失效与回填同拍	失效占用目录写口，回填放弃
Load 发出后 epoch 改变	回填 epoch 与当前 epoch 不等则不安装，避免安装失效前的旧数据

放弃回填时，Load 和写直达 Store 继续完成，后续访问该字时重新从后备存储读取。

`coh.inv_valid/inv_addr` 与 8 bit epoch 仅在共享 L2 实验路径使用。远端写入按组索引清除该组两路 valid，每路执行一次目录写入。LSU 在 Load 发出时采样 epoch，回填时比较当前版本。主线单核和 AMP 将一致性输入接零。

访存单元 LSU 与访存控制

快慢路径及阶段对齐

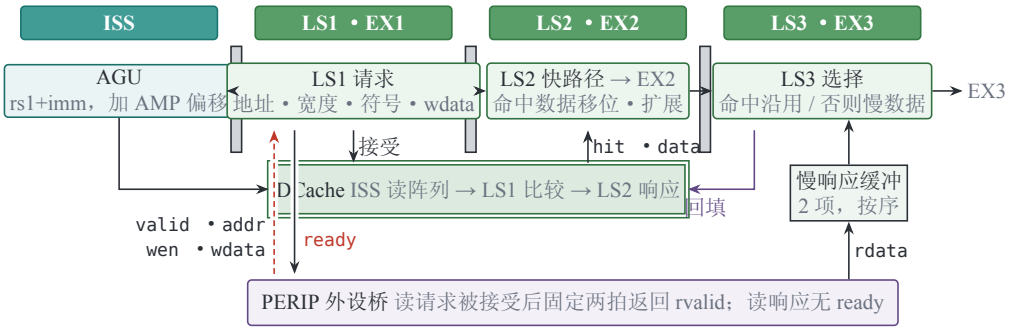
LSU 由 LS1、LS2、LS3 组成，与 EX1、EX2、EX3 对齐；ISS 中的提前 AGU 和 DCache 查表位于正式访存请求之前。阶段职责见表 14，数据通路见图 12。

表 14: LSU 阶段及请求、响应对齐

阶段	地址、数据与控制行为
ISS 提前查询	形成有效地址，加入 AMP 数据偏移；驱动两路 Tag/Data 及 LRU 同步读口
LS1 / EX1	接收地址、宽度、符号属性及 Store 数据；向 PERIP 发出正式请求，握手后保存请求和 victim
LS2 / EX2	提供 DCache hit 与格式化快数据；Store 命中时更新缓存副本；确定 Load 的 EX2/EX3 就绪时机
LS3 / EX3	按序取得桥返回或慢响应缓存队首，完成慢路径格式化；按条件将原始字回填 DCache
后续 EX4 / WB	传递已选定的结果及退休信息，最终顺序写回

Load 命中时在 EX2 取得 `rdata_fast`，结果可用于旁路；EX3 继续传递已获得的值。未命中或访问 MMIO 时，EX3 使用 `rdata_slow`。两条路径均先按地址低位移位，再对 byte/half 截取，按 LB/LH 或 LBU/LHU 进行符号/零扩展；LW 使用完整字。Load 的结果可用时机由命中情况决定。

DCache 通过 EX2 提前提供命中结果，缩短 Load-use 延迟。LS1 在 LS2 命中结论产生前向桥发出 Load，因此命中时仍执行一次外部读取。EX3 按序消费慢响应，命中时沿用快路径结果，miss 时选择慢数据并按条件回填。已接受请求通过 LSU 流水与慢响应缓冲推进，后续发射由相关性和桥背压控制。



valid & !ready 冻结 EX1 及更年轻级，直到同一请求握手。Load 命中时桥读仍完成，LS3 丢弃慢数据；Store 在 LS1 写直达入桥，LS2 命中时同步更新 DCache 副本；慢响应缓冲容纳流水冻结后仍返回的两个读。

图 12: 三阶段 LSU 与快慢响应路径

请求握手、背压与慢响应保持

PERIP 请求以 `valid && ready` 为接受事件，`wen` 仅表示读写属性。LSU 在存在 LS1 访存、且没有更老长操作暂停时持续置 `perip.valid`；桥不能接收时，EXU 由 `perip.valid && !perip.ready` 产生 `backend_stall`，保持执行流水及上游内容，直到同一请求完成握手。缓存请求、Store 入桥及读响应标签均在接受事件发生时更新。

LSU 传给请求有效逻辑的 `stall` 只包含更老的长操作暂停，桥背压则经 `ready` 本地合并到流水使能中，避免 valid 与 ready 经全局停顿形成组合依赖环。长操作暂停时 `perip.valid` 降低，相关地址/宽度选择维持旧响应的对齐；桥背压保持尚未接受的 LS1 请求。

`student_top/perip_bridge` 路径约定：读请求一旦被接受，数据经过 BRAM 读取和输出寄存，在两拍后以 `perip.rvalid` 返回，正好对应 LS3；MMIO、周期计数、加速器和 mailbox 的响应也对齐到该时刻。PERIP 没有读响应 ready，已经发出的读不会因为核停顿而停止返回。LSU 因而设置两个按序数据槽：若 LS3 可消费且队列为空，本拍返回直接使用；否则返回入队，恢复后优先消费队首。

两个槽位容纳长操作冻结流水后仍会陆续返回的两个已接受读。

Load 命中预测与就绪时机修正

为避免 DCache tag 比较直接串回 ISS/GPR 使能路径，Load 在 EX1 的生产时机按 EX2 命中预测。相关网络据此允许满足条件的后继指令继续推进；EX2 再用寄存的 `cache_hit` 修正结果可用时机：命中为 `WHEN_ABS_EX2`，miss 为 `WHEN_ABS_EX3`。图 13 给出紧邻消费者在命中与未命中时的停顿周期。

命中预测只决定 Load 自身何时可被旁路，紧邻消费者不会因此提前带走未确认的数据。Load 位于 EX1 时相对生产码为 00011，而所有实际使用的源操作数需求均为 NEED_ISS_LATE 或 NEED_ISS_EARLY，按位相与非零，消费者先在 ISS 停顿一拍。下一拍 Load 进入 EX2，ISU 看到的已是按 `cache_hit` 修正后的时机：命中时相对码为 00001，消费者发射并经旁路取得数据，共停顿一拍；miss 时相对码为 00011，消费者再停顿一拍，待 Load 在 EX3 选出慢路径数据后发射，共停顿两拍。修正发生在消费者离开 ISS 之前，因此预测错误只增加停顿，不需要撤销已发射的指令。地址基址另受 ISS 早期 AGU 需求约束。

EXU 仍保留 `load_miss_replay` 作为保护路径：若 EX2 是可缓存窗口内的 miss Load，且 EX1 中较年轻指令的 rs1 或 rs2 实际依赖该 Load，则以该消费者 PC 重定向，丢弃消费者与更年轻的前端内容，较老 Load 沿慢路径继续完成。按生产/消费时机编码，上述 ISS 互锁已覆盖这一条件，普通写后读相关不会触发该路径；`PipelineStats` 单独统计其触发次数，可用于在仿真中核对。MMIO Load 通过慢路径完成，不参与该保护路径。



图 13: Load 命中与未命中时的相关时序

外设桥写队列与逐字节转发

四项 Store Buffer 位于 `perip_bridge`，每项保存数据窗口内地址、32 bit 写数据及 2 bit 访问宽度。DRAM Store 握手后进入 FIFO，头尾指针与计数分别管理入队和出队；同地址 Store 也保留顺序条目，不在入队时合并。队列非空时头项连续排入一级 BRAM 写命令寄存器，再由独立写端口写入数据 BRAM。写队列排空需要同时满足 FIFO 为空和暂存写无效。同拍出队与入队可重叠，队列满时也可以借助本拍出队接收下一 Store。BRAM 读端口独立服务 DRAM Load，不要求等待整个队列排空。

接受 Load 时，桥对同一个对齐字地址的较老 Store 做快照：先取已暂存写命令，再按 FIFO 从最老到最新扫描。每个匹配条目只覆盖其有效字节，较新的较老 Store 对同一 byte lane 的值优先。转发 mask 和数据随读响应标签经过两拍流水，在返回时与 BRAM 原字合并。若掩码扩展为 32 bit 的 F，则

$$D_{\text{load}} = (D_{\text{BRAM}} \wedge \neg F) \vee (D_{\text{forward}} \wedge F). \tag{5}$$

未被较老 Store 覆盖的字节仍来自 BRAM；这允许 Load 读取队列中多条 byte/half Store 拼接出的最新完整字，同时避免依赖 BRAM 同址读写返回旧值还是新值的器件差异。结构与示例见图 14。

例如，初始对齐字为 `0x11223344`，按顺序执行 `SB 0xAA, 1(A)`、`SH 0xBBCC, 2(A)`，随后执行 `LW 0(A)`。无论这两次写仍在 FIFO、正在暂存还是已经落地，按字节转发所要求的返回值都是 `0xBBCCAA44`。

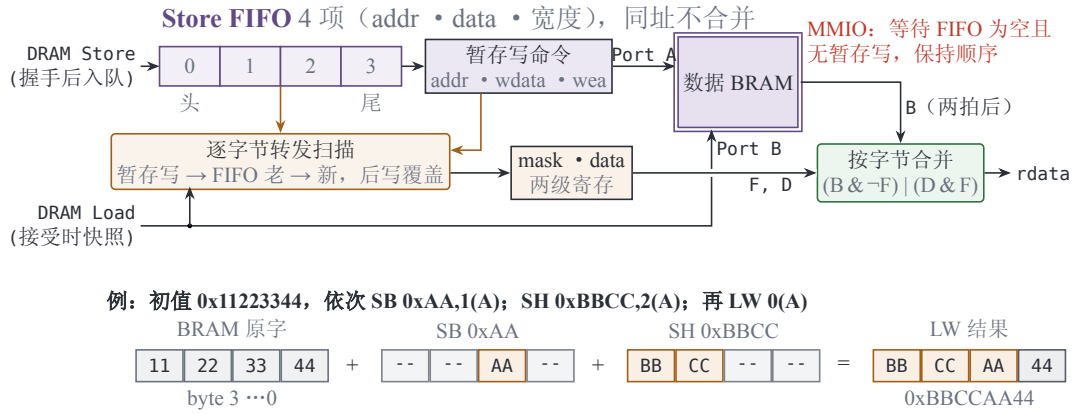


图 14: 外设桥写队列与逐字节 Load 转发

MMIO 顺序、FENCE 与中断排空

MMIO 读写绕过 DCache，并在桥中等待 Store FIFO 为空且暂存写无效，保证不会越过尚未落地的较老 DRAM Store。UART TX 还要求 `console_tx_ready`，只有最终接受事件才触发发送；加速器请求也只在桥接受后转发。

FENCE 停留在 ID，直到 `LSU.idle` 与 `perip.idle` 同时成立。前者检查 LS1–LS3 是否仍有访存、慢响应缓冲是否为空以及是否正在发请求；后者检查写 FIFO、暂存写、请求、加速器返回和两级读响应标签是否全部空闲。排空后 FENCE 作为不写寄存器的指令继续推进，等待期间阻塞年轻指令发射。加速器内部计算的完成状态由软件读取引擎状态寄存器判断。机器定时器中断还要停止新发射，并等待执行、写回及长操作排空，再通过寄存的空闲判断进入 trap；它复用访存空闲信息，但恢复 PC 来自最后退休指令的真实下一 PC。`FENCE.I` 仅共用 FENCE 排空语义，ICache 的事务 flush 与缓存内容失效仍是不同操作。

本节对应 `core/scsrc/lsu/dcache.scala`、`core/scsrc/lsu/lsu.scala`、`core/scsrc/exu/EXU.scala`、`soc/jyd/perip_bridge.sv` 及三类 DCache BRAM 包装，路径均相对 `rtl/pipeline/`。现有 `pipeline-dcache-test` 包含同组双项驻留、LRU、部分写、失效、epoch、查表保持及回填冲突等定向和随机检查；`pipeline-smoke`、程序回归及 IRQ 测试用于覆盖系统顺序与暂停组合。

写回单元 WBU

WBU 是顺序提交点，选择普通执行结果、乘法结果或除法结果写回通用寄存器堆。除法未完成时，会阻塞 WB 阶段，以确保顺序提交。x0 的恒零约束由寄存器堆实现。仿真环境以 WBU 的有效提交作为 guest 指令计数依据，并在该处获得访存和环境指令信息。

冒险处理

数据冒险与旁路网络

旁路与互锁是顺序流水线处理 RAW 相关的基本手段 [7, 8]。本设计将数据冒险主要分开为回答三个问题：**生产者什么时候能生产出可旁路的数据；消费者什么时候需要生产者的数据；被旁路的数据在哪里。**

为**解决“生产者什么时候能生产出可旁路的数据；消费者什么时候需要生产者的数据”两个问题**，本设计在 IDU 阶段，根据不同的指令类型，给出了各个指令的**生产时机**和**消费时机**。生产时机由 5 bit 的 PROD_ABS 温度计码表示。从低位到高位依次对应 EX1、EX2、EX3、EX4 和 WB 检查点；某一位为 1 表示结

果到该检查点仍未就绪，为 0 表示此时已经可以旁路。因此，连续的低位 1 越多，代表生产者产生结果越晚。本文以 PROD_ABS_* 表示生产码，对应代码中的 WHEN_ABS_*；各类指令的生产时机如表 15 所示。

表 15: IDU 生产时机编码

编码	温度计码	结果可旁路时机	对应的指令
PROD_ABS_EX1	00000	EX1 末期	普通整数运算、移位和比较指令，JAL/JALR、CSR 指令、AUIPC/LUI
PROD_ABS_EX2	00001	EX2 末期	所有 Load 的命中预测；CRC8
PROD_ABS_EX3	00011	EX3 末期	EX2 确认 miss 后的 Load 慢路径
PROD_ABS_EX4	00111	EX4 末期	暂无生产者
PROD_ABS_WB_EARLY	01111	WB 早期	MUL/MULH/MULHSU/MULHU
PROD_ABS_WB_LATE	11111	WB 完成	DIV/DIVU/REM/REMU

其中，PROD_ABS_EX1，与CONS_EX4，等效为立刻可以生产/最晚才需要消费，故不生产数据/不消费数据的指令，统一被编码为PROD_ABS_EX1，与CONS_EX4。

消费码记为 CONS_*，对应代码中的 NEED_*；AGU 在 ISU 中将访存基址的需求覆盖为 ISS 早期。各类操作数的消费时机如表 16 所示。

表 16: 消费者需求时机（含 ISU 的 AGU 覆盖）

编码	温度计码	消费时机	对应的指令
CONS_ISS_EARLY	11111	ISS 早期	Load/Store 类指令的地址 base rs1
CONS_ISS_LATE	11110	ISS 末期	ALU、branch、CSR 源寄存器、Store 数据及普通操作数
CONS_EX1	11100	EX1	暂无消费者
CONS_EX2	11000	EX2	未分配消费者
CONS_EX3	10000	EX3	未分配消费者
CONS_EX4	00000	EX4	无需读取的源操作数占位

IDU 给出的 PROD_ABS_* 以生产者自身的流水过程为绝对时间坐标，而相关判断发生时，消费者正位于 ISS，生产者可能已经处于 EX1 至 WB 中的任意一级。因此，ISU 需要结合生产者的当前位置，将绝对未就绪温度计码平移成“从

当前消费者出发还要等待多久”的相对未就绪温度计码。本文将平移后的六种取值命名为 PROD_REL_*，如表 17 所示。

表 17: 生产者相对未就绪温度计码

编码	温度计码	从当前消费者观察的结果可用时机
PROD_REL_ISS_EARLY	00000	当前 ISS 早期已经可用
PROD_REL_ISS_LATE	00001	当前 ISS 末期可用
PROD_REL_EX1	00011	消费者进入 EX1 时可用
PROD_REL_EX2	00111	消费者进入 EX2 时可用
PROD_REL_EX3	01111	消费者进入 EX3 时可用
PROD_REL_EX4	11111	消费者进入 EX4 时可用

为**解决“被旁路的数据在哪里”的问题**，ISU 使用提前比较结果寄存与 EX1 当拍比较生成 where。对于 rs1 和 rs2，ID 提前比较 EX1–EX4 的目的寄存器，寄存结果在 ISS 对齐至 EX2–WB，并补充 EX1 的当拍比较。某一级的匹配条件可以表示为 `match_s = rs_ren && stage_valid && rd_wen && (rd != x0) && (rs == rd)`：只有消费者确实读取该源寄存器、该级指令有效且会写非零目的寄存器时，才认为存在 RAW 相关。各级匹配结果按照表 18 的布局组成 6 bit 的 where，其中 ISS 保留位置零，低五位表示当前有效旁路位置。rs1 和 rs2 各自维护一份 where。

表 18: where 相关位置向量的位布局

位	对应位置	置 1 条件
where[5]	ISS	保留位，恒为 0
where[4]	EX1	源寄存器与 EX1 的有效目的寄存器匹配
where[3]	EX2	源寄存器与 EX2 的有效目的寄存器匹配
where[2]	EX3	源寄存器与 EX3 的有效目的寄存器匹配
where[1]	EX4	源寄存器与 EX4 的有效目的寄存器匹配
where[0]	WB	源寄存器与 WB 的有效目的寄存器匹配

如果多条在途指令连续写同一个寄存器，其中可以同时有多个 bit 为 1。获得 where 后，设绝对生产码为 PROD_ABS[4:0]，从高位到低位依次对应 WB、EX4、

EX3、EX2、EX1，则 ISU 针对生产者的五种可能位置生成表 19 中的候选码。随后，PROD_REL 使用 where 选通相应候选码并按位或，得到最终的 PROD_REL。

表 19: 绝对生产码到相对未就绪码的平移

生产者位置	候选相对码的生成方式	含义
EX1	`[PROD_ABS[3:0], 1'b1]`	末位补 1，表示 EX1 结果无法旁路到 ISS 早期
EX2	`[1'b0, PROD_ABS[3:0]]`	相对 EX1 情况前进一级
EX3	`[2'b00, PROD_ABS[3:1]]`	移去两个已经经过的检查点
EX4	`[3'b000, PROD_ABS[3:2]]`	仅保留尚未经过的检查点
WB	`[4'b0000, PROD_ABS[4]]`	普通结果已经就绪；除法未完成时保留末位未就绪标志

生产者每向后推进一级，候选相对码便右移一位，移去一个已经经过的检查点，并在高位补 0。例如，普通 ALU 指令的绝对码为 00000，当它位于 EX1 时得到 00001，即 PROD_REL_ISS_LATE；该值与后继普通指令的 CONS_ISS_LATE（11110）按位相与为 0，因此可以直接旁路。LW 的绝对码为 00001，当它位于 EX1 时得到 00011，即 PROD_REL_EX1；与 CONS_ISS_LATE 按位相与的结果非零，因此后继指令停顿一周期。下一周期 LW 进入 EX2，相对码在命中时变为 00001，此时即可旁路；miss 时改为 00011，消费者再停顿一周期，待 EX3 慢路径数据可用后发射。类似地，AGU 基址使用 CONS_ISS_EARLY，即使紧邻的普通 ALU 生产者已经得到 PROD_REL_ISS_LATE，两者按位相与仍非零，所以互锁逻辑会准确插入一个周期停顿。

互锁判断与旁路选择共用同一份 where。ISU 将消费者的 CONS_* 与 PROD_REL_* 按位相与；结果非零表示生产者无法赶上消费时机，需要触发 interlock。与此同时，各级生产者根据自身绝对生产码生成 bypass_mask，其位布局为 6 bit（ISS 保留位为 0），低五位与下表对应，判断方式如表 20 所示。

表 20: 旁路数据就绪掩码

位置	掩码位	置 1 条件
EX1	`bypass[4]`	EX1 有效、写寄存器，且 PROD_ABS[0] 为 0
EX2	`bypass[3]`	EX2 有效、写寄存器，且 PROD_ABS[1] 为 0
EX3	`bypass[2]`	EX3 有效、写寄存器，且 PROD_ABS[2] 为 0
EX4	`bypass[1]`	EX4 有效、写寄存器，且 PROD_ABS[3] 为 0
WB	`bypass[0]`	WB 写回结果有效且写寄存器

将相关位置与就绪掩码按位相与，即得到当前真正可用的旁路源。如果没有任何 bit 置 1，旁路选择为 FWD_NONE；否则按照 EX1、EX2、EX3、EX4、WB 的顺序进行优先选择，生成独热的 FWD_EX1 至 FWD_WB。由于处理器顺序单发射，位置越靠近 EX1 的生产者越年轻，这一优先级可以保证多条在途指令写同一寄存器时总是选择最新值。

旁路数据通路分两次完成操作数修正。第一次位于 ISS：ISU 使用独热选择信号，在寄存器堆读数以及 EX1、EX2、EX3、EX4、WB 五路结果之间选择，得到 rs1_val_iss 和 rs2_val_iss。选择 FWD_NONE 时保留寄存器堆读数，否则用对应流水级的结果覆盖。

消费者进入 EX1 时，advance_where 将尚未解决的相关位置随生产者一起向后移动一级，形成 where_ex1。已经在 ISS 取得的生产者以及所有更老的匹配项会被清除；尚未就绪的更年轻生产者则继续保留。EX1 使用新的 where_ex1 与当前 bypass_mask 再次计算旁路源：若更年轻的结果此时变为可用，ISU 用该结果覆盖 ISS 已经取得的值；否则保留 rs1_val_iss 和 rs2_val_iss。这一两级旁路结构既能尽早取得现有结果，又能避免较老的结果覆盖稍后才就绪的年轻结果。

控制冒险

EX1 的 BRU 计算真实 next PC，HZU 将误预测标记、真实目标及预测器更新信息寄存一拍。重定向清空前端元数据和队列、ID/ISS 与 ISS/EX1 的错误路径内容，并通过 `irom.flush` 丢弃旧取指事务。定时器中断和 Load miss 保护路径也可触发前端重定向；中断恢复入口优先于普通分支，普通分支目标优先于保护路径给出的消费者 PC。按生产/消费时机编码，普通写后读相关由 ISS 互锁处理，不经过该保护路径。

结构冒险

数据请求共享 PERIP 握手口，`valid && !ready` 使流水线保持请求，避免重复或丢失访问。桥中的四项 FIFO 通过独立 BRAM 写口连续排出，DRAM Load 使用独立读口并合并尚未落地的较老 Store 数据；MMIO 则等待写队列和暂存写排空。FENCE 在 ID 等待 LSU 与桥同时空闲；EBREAK 退休后冻结流水线。

自研 miniTensor 加速器

miniTensor 接在 CPU 的 CV-X-IF 通道上，不是挂在总线上的外设：CPU 只通过 8 个 MMIO 寄存器写指令和两个操作数，shim 合成 issue→register→commit→result 事务，DMA 在统一缓冲与 scratch/BRAM 之间搬数。三个限制决定了它的结构：无 FPU，全整数；无数据 Cache，CPU 与 DMA 必须看到同一物理视图；窗口只有 4 KiB，命令就得尽量简单。图 15 给出 shim、64 项命令队列、4×4 阵列、向量单元、epilogue、UB 与 DMA 的关系。

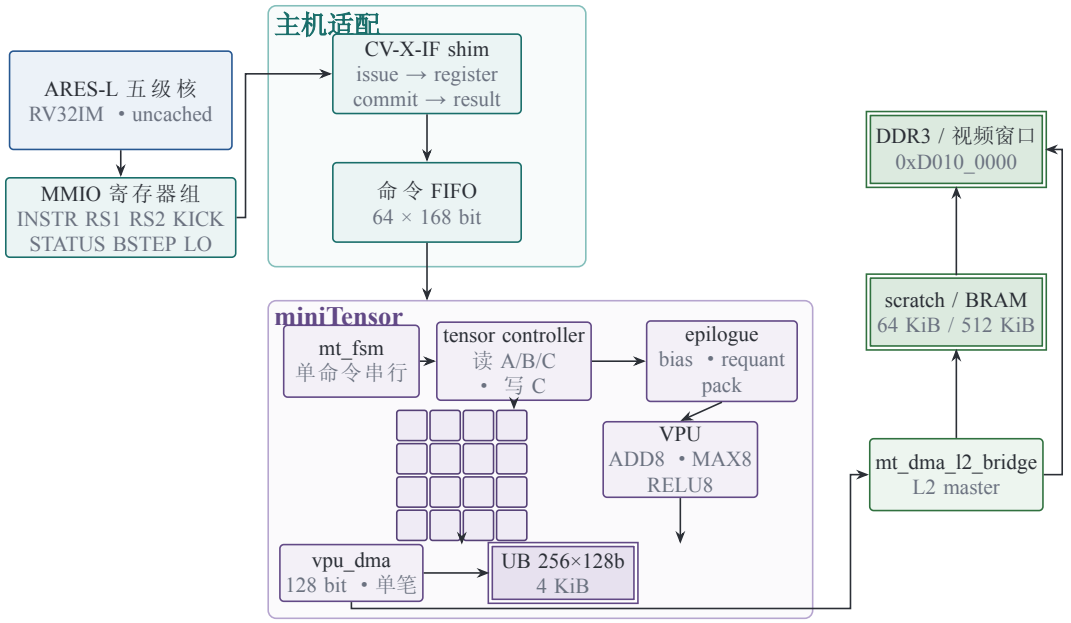


图 15: miniTensor 的模块关系：主机适配、命令队列、统一缓冲、4×4 阵列、向量单元、epilogue 与 DMA

UB 每行 128 bit，正好 16 个 INT8 或 4 个 INT32；A/B tile 各占一行，C tile 占四行，4 拍把 $k = 0, \ldots, 3$ 的乘积累加到 16 个 32 bit 累加器，`accumulate` 位选择初始化或累加。所有指令为 custom-0：`BURST` 用 rd 当重复次数，`GEMM2D` 用 nu×nj 展开二维网格，`CONVGROUP` 把 LOAD 与网格合成一条。图 16 给出 GEMV 映射：A 行 0 放 $x[4t..4t{+}3]$、行 1–3 补零，B 放 [组][k 组][4k][4out] 权重，C 只取行 0，沿

t 累加。这样一来，1×1 卷积就是 GEMV，不需要第二套数据通路，代价是阵列利用率约 1/4。

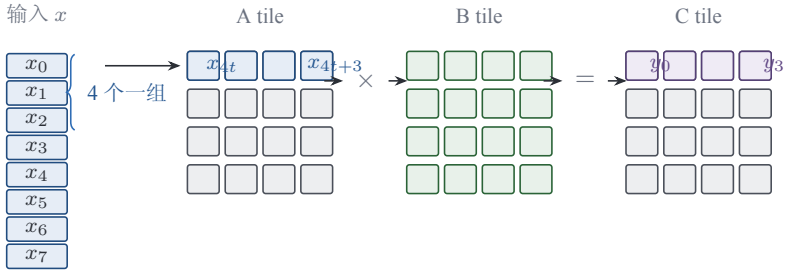


图 16: GEMV 到 4×4 阵列的映射：A 行 0 放 4 个输入元素，行 1–3 补零，C 只取行 0 并沿 t 累加；1×1 卷积即 GEMV

基线每帧 17,184 条命令、约 7.27 M 周期，加速器 busy 只有 7.7%；把外层循环搬进 shim 后降到 1,936 条、约 1.31 M 周期，busy 升到 39%。b5 post-route：CLMA 10,229/30,075（35%）、FF 24,404（11%）、DRM 261/415（63%）、APM 62/740（9%）；b5−b4 为 +6,276 LUT、+5,067 FF、+48 APM、+4.5 DRM；27 MHz WNS +12.85 ns，推算上限约 41.9 MHz。验证采用命令级单测、逐层整数 golden 与板端串口：手势 287 帧 INT8 100%、板端 8/8，A0 40 token 0/40。它的优势可以概括为“同一阵列覆盖两类负载、命令化把循环搬进硬件、资源增量可控”。表 21 给出关键几级优化：命令数从 17,184 降到 1,936，周期从 7.27 M 降到约 1.31 M。

表 21: miniTensor 的性能演进（手势网络，golden 均为 0 字节差异）

版本	关键改动	命令/帧	周期/帧
B7b	基线：CPU 逐条 push	17,184	7.27 M
P1b	GEMM 突发（BURST）	6,720	4.85 M
P1d	pad/pool 词宽化（纯 C）	6,032	2.00 M
P1f	二维 GEMM 突发（GEMM2D）	3,120	1.42 M
P1g	LOAD 与二维突发合并（CONVGROUP）	1,936	1.35 M
P1h	状态字加 FULL，满才等	1,936	1.23 M

2.3 软件系统介绍

2.3.1 软件整体介绍

目标端包括 AbstractMachine 裸机运行时与 CoreMark，以及独立的 RT-Thread Nano 自检和 MSH 固件；主机端提供镜像拆分、Verilator 驱动和模块测试脚本。核生成配置关闭 DPI-C/difftest 仿真钩子。裸机仿真仍可通过独立 DPI 存储器模型运行，退休观察使用显式 `retire_*` 接口；RTOS 板级仿真使用推断 BRAM 和实际 UART RTL。

2.3.2 软件各模块介绍

仿真平台

Verilator [9] 将 RTL 编译为 C++ 模型，测试台负责时钟、复位、镜像和结束条件。`rtl/pipeline/test` 提供 smoke、ISA、CoreMark、Cache、乘除法、IRQ、UART、AMP 等入口；波形和日志按对应构建配置生成。`CONFIG_SIMULATION=false` 关闭核内仿真插桩，与 Verilator 构建中的独立 DPI 存储器接口是不同开关。

软件测试环境

主机环境为 Ubuntu 24.04.3 LTS、Verilator 5.046 和 Chisel 7.0.0-M2；CoreMark 等目标程序由 `riscv64-unknown-elf-gcc` 13.2.0 按 `rv32im_zicsr/ilp32` 编译；FPGA 实现使用 Pango PDS 2025.2-SP3。测试程序与裸机运行时环境共同编译，链接至处理器能够访问的地址空间，再由脚本生成 BIN 和 IROM/DRAM HEX。

针对赛题的内存空间设计编写了专用的链接脚本以及入口函数，为编译程序提供最基本的运行时环境。目标端地址空间与硬件采用相同的 IROM、DRAM 和外设映射，见第二部分的 SoC 地址空间。

硬件仿真环境

硬件仿真由对应顶层、存储模型、时钟/复位和测试台组成；ISA 程序检查 EBREAK 退休及 `a0`，CoreMark 检查输出 CRC 和通过标记，模块测试检查缓存、算术、外设与中断协议。前身 Mundus 的逐指令参考模型与 BPU 差分结果见附录 B。

测试程序

测试入口覆盖 `pipeline-smoke`、`pipeline-isa`、CoreMark 各镜像、DCache 与乘除法单测、多核 mailbox/存储实验及 RTOS/IRQ/UART。cpu-tests 包含 36 个程序级回归用例。

工程模块组织

ARES 的相关模块组织如下，构建产物位于 `build/`。

```
ARES/
|-- rtl/pipeline/core/scsrc/ Chisel core, DCache and CRC8
|-- rtl/pipeline/core/vsrc/ Generated core and handwritten wrappers
|-- rtl/pipeline/gen/ mill build and core configuration
|-- rtl/pipeline/soc/ jyd, amp, mc, ip and pango
|-- rtl/pipeline/board/ Clock/reset and board tops
|-- rtl/pipeline/test/ RTL testbenches and Verilator drivers
```

```
|-- rtl/accel/ Matrix, state and list engines
|-- fpga/pango/pipeline/ PDS flow and constraints
|-- tests/abstract-machine/ Bare-metal runtime and linker scripts
|-- tests/benchmarks/coremark/ CoreMark and port
|-- rtthread-nano/rt-thread/bsp/ARES/ RTOS BSP
|-- scripts/ Firmware/image/test tools
`-- build/ Local generated images, models and logs
```

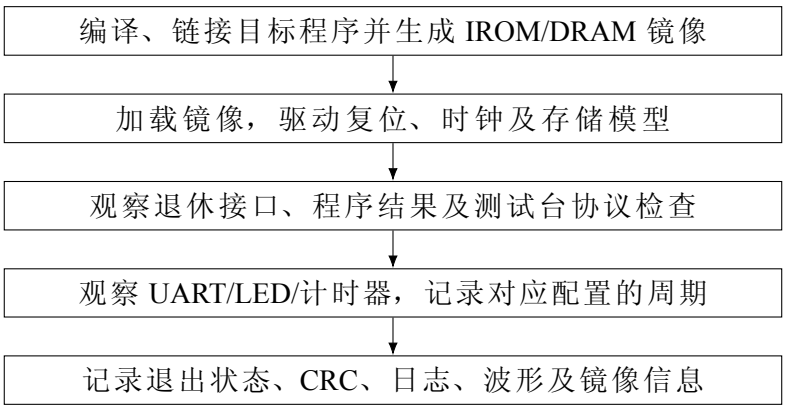


图 17: 主机仿真框架的运行与检查流程

表 22: 软件模块的关键输入输出

模块	关键输入	关键输出
运行时与镜像	链接布局、程序及初始数据	入口与 IROM/DRAM 镜像
仿真驱动	镜像、复位、时钟	波形、日志与运行状态
退休观察	退休有效、PC、GPR 写回及 EBREAK	程序结束条件及指令计数
BPU 差分（前身）	RTL 与参考模型的预测及更新事件	模型匹配率与预测准确率
IO 模拟	LED、SEG、计时控制	灯组、数码管和运行时间

2.3.3 应用一：语言模型部署

Qwen2.5-0.5B-Instruct [12] 有 24 层、$d = 896$、494M 参数，FP16 权重约 1 GB、INT8 仍约 494 MB；片上窗口只有 512 KiB。把 0.5B 搬到 DDR 需要 64–128 lane 的 GEMV 和 BPE 词表，是另一条路线；我们选择把能力压到 10^5 参数级、让它常驻片上：Qwen 提供语料与语言能力，板上运行 H112。

四个形态各管一件事（表 23）：H112 140,742 参数 / 152,056 B，是当前部署；H88 107,000 / 118,168 B，适配 128 KB loader；A0 56,480，用来做 CPU 与加速器的

同模型对照；QA 10,475 条 / 638,076 B，留出模板 94%。A0 的 CPU 路径用 int16 Q15、17 char/s；miniTensor 把 GEMV 映射为 1×1 卷积，int8、103 char/s @27 MHz、0/40 golden。6 倍提升主要来自“把 GEMV 变成阵列友好的形状”，代价是 INT8 激活的文本质量，下一步做 QAT。图 18 给出从 Qwen 语料到两条执行路径。

表 23: 语言模型与知识库的四个形态

代号	类型	参数	镜像	定位
A0	char-GRU（Emb64→GRU96→Linear65）	56,480	62 KB	CPU/加速器同模型对照
H112	词级 seq2seq GRU（V198/E48/H112/rank24）	140,742	152,056 B	当前部署
H88	词级 seq2seq GRU（轻量版）	107,000	118,168 B	128 KB loader
QA	检索式知识库	10,475 条	638,076 B	留出模板 94%

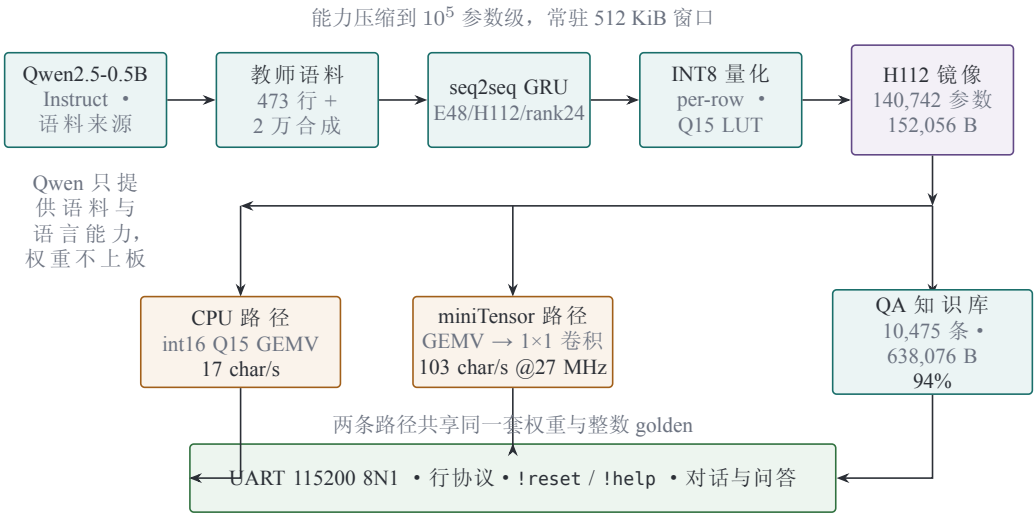


图 18: 从 Qwen 语料到 H112，再到 CPU 与 miniTensor 两条执行路径

串口采用 115200 8N1 行协议；H112 的 20 token/s 是 50 ms/token 的输出节流，不是模型算力。上电先位流后固件；128 KB loader 装 H88，512 KiB 窗口装 H112 与 QA。这里的优势不是“把 0.5B 搬上板”，而是把 Qwen 的能力压到 152 KB、常驻片上，并用同一套权重跑出 CPU 与加速器两条可对比的路径。

2.3.4 应用二：手势识别

OV5640 输出 720p，取中心 640×480，按 2:1 抽成 320×240 灰度，写入 `0xD010_0000`；固件放一个 13×11 格的固定取景框（约 240×165 px），把框内区域 letterbox 到 32×32，量化后送入 miniTensor 的四层 CNN；每帧做 argmax、裕度门限和 3 帧确认，结果经 UART 以 10 Hz 输出。图 19 给出从摄像头到网页的完整链路。

网络为 conv3x3 4→8、8→16、16→16，接 1×1 head 16→8；3,920 参数、约 745K MAC。287 帧（石头 68 / 布 109 / 剪刀 110）上 INT8 全量 100%，浮点增强

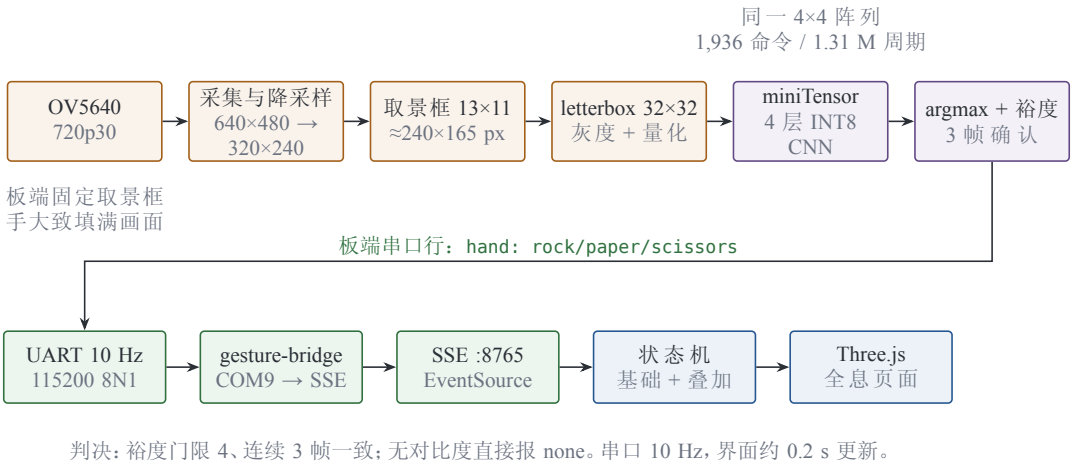


图 19: 手势识别链路：OV5640、ROI、32×32 量化、miniTensor CNN、UART 与网页

92%，板端 8/8 与离线一致。每帧 1,936 条命令、约 1.31 M 周期：100 MHz 仿真 13.1 ms，27 MHz 板级 48.5 ms；串口 10 Hz，连续 3 帧确认使界面约 0.2 s 更新。

表 24: 手势数据集与精度

项目	结果
数据集	287 帧（石头 68 / 布 109 / 剪刀 110）
3-NN 留出集	100%（随机 33%）
浮点网络	干净 100%，增强后 92%
打包后 INT8	全部 287 帧 100%
板端	8/8 与离线逐字节一致

它的好处是“小阵列 + INT8 也能在 10 Hz 下跑完整 CNN，并与检测器共用同一加速器”。

2.3.5 演示层：上位机

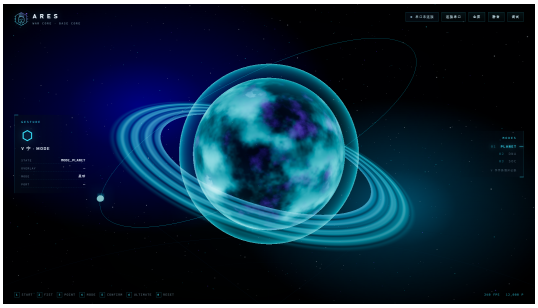
上位机把串口输出翻译成页面状态：gesture-bridge 独占 COM9，把板端手势行映射为 `FIST/START/MODE`，经 SSE 推给页面；页面按 LineParser → CommandBus → StateMachine → EffectDirector → Three.js 驱动全息效果。33 个单元测试、15/15 浏览器端到端通过，生产包约 608 kB；RTX 5060 Laptop / 2560×1600 / HIGH 下实测 240 FPS。图 20 为生产构建实拍。

2.4 系统特色：加速器、多核协作与实时调度

ARES 将顺序流水线核、专用计算引擎和系统软件通过明确的调用接口连接起来。CRC8 使用核内自定义指令，矩阵、状态机和链表使用独立 MMIO 引擎；AMP 将计算状态分散到各 hart 的私有存储，通过 mailbox 汇总结果；单核 RT-Thread



(a) 握拳：压缩/蓄力



(b) 剪刀：星球模式

图 20: 手势经串口桥接驱动网页全息效果（生产构建实拍）

Nano 以机器定时器中断驱动抢占和线程切换。下面分别说明这些接口及执行过程，性能与验证结果见第 3.3 节。

从加速方式看，ARES 同时使用核内自定义指令与协处理器两条路径。CRC8 与三类 MMIO 引擎面向控制密集、数据量小的固定算法；miniTensor 面向卷积、GEMV 等张量负载。两者共享 uncached 数据视图和 MMIO 语义，但接口不同：前者进入流水线，后者经 CV-X-IF shim 与 DMA 工作。

2.4.1 CRC8 与三个 MMIO 引擎的软硬件协同

构建接入与调用接口

`scripts/build_coremark_crc.py` 将 CoreMark 复制到构建目录，在副本中加入驱动翻译单元，并只对原定义所在目标文件进行符号重命名。例如，将 `core_util.o` 的 CRC 定义改名为 `*_orig`，其余调用点仍使用 `crcu8/crcu16/crc16/crcu32`，由新驱动提供同名函数。矩阵、状态机和链表也以相同方式替换函数入口，保留软件参考实现。这样，基准算法的调用关系与结果校验继续使用原接口，硬件选择由构建选项决定。

表 25: 加速路径与软件职责

路径	硬件工作	软件工作
CRC8	对一个字节执行八步 CRC 递推	传入字节与 16 bit CRC，按低字节优先组合更宽输入
矩阵	固定 9 阶矩阵的标量、向量、矩阵和位提取乘积及校验	拷贝 A/B，设置参数，读取局部 CRC 并并入外层 CRC
状态机	两次 token 扫描、扰动/还原、状态计数及 CRC	拷贝数据块，写入 seeds、step 与初始 CRC，读取最终 CRC
链表	find、reverse、move-to-front 热循环	装载节点影子，读回顺序，继续排序、移除/恢复和 CRC 遍历

三个引擎分别位于 0xA0001000、0xA0002000 和 0xA0003000，由 pipeline_accel.sv 译码 4 KiB 页。CPU 通过 volatile MMIO 访问寄存器；写入数据和参数后，写 CTRL 的 bit0 启动，轮询 STATUS 的 bit1，完成后读取结果。STATUS 的 bit0 表示 busy，启动时清除旧 done，计算完成后置 done。数据区由空闲时的 host 与忙时的引擎分时使用。

MMIO 请求与计算任务分开：引擎接受寄存器访问，返回端在下一拍形成响应，包装层再寄存一拍，与外设桥的两拍读响应对齐；计算则由引擎内部状态机推进。操作数从 CPU 经 MMIO 拷入引擎私有数组，完成后仅取回结果或节点影子。该过程不使用 DMA，MMIO 也不进入 DCache，因此无需在 CPU 与引擎之间维护共享缓存副本。两类加速路径的调用关系如图 21 所示。

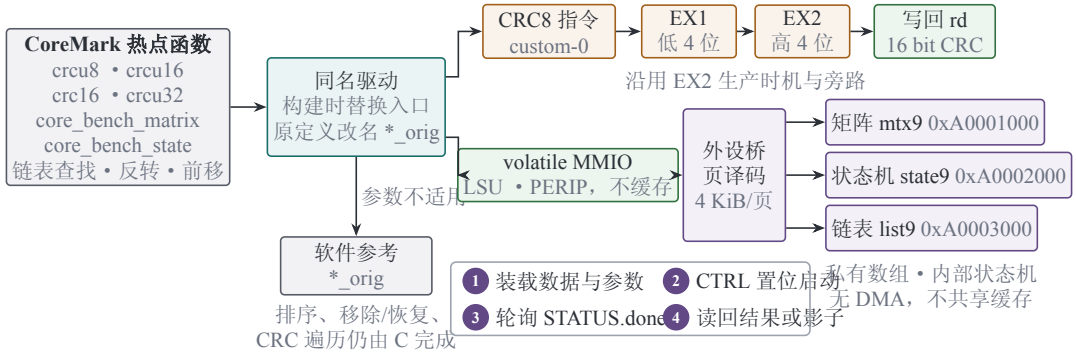


图 21: CRC8 指令与 MMIO 引擎的调用关系

CRC8：将八步递推拆为两个执行阶段

CRC8 指令每次处理 8 bit 输入数据，维护 16 bit CRC 状态，名称中的 8 指输入粒度。其编码与寄存器语义为：

```
.insn r 0x0b, 0, 0x00, rd, rs1, rs2
// rs1[7:0] = data, rs2[15:0] = input CRC
// rd[15:0] = updated CRC, rd[31:16] = 0
```

即 opcode 为 custom-0 的 0x0B，funct3 和 funct7 均为零。单个 bit step 取数据最低位与 CRC 最低位异或，根据结果对 CRC 异或 0x4002，右移后将该异或结果放到最高位；数据也右移一位。连续执行八次即得到与 crcu8() 相同的结果。

核内 CRC8Ops.step4 一次展开四步：EX1 处理输入低半字节并保存部分 CRC，EX2 处理高半字节，将 16 bit 结果零扩展后送回原写回通路。译码将其生产时机标为 EX2，沿用现有旁路和相关互锁。软件 crcu16() 先处理低字节、再处理高字节；crcu32() 先处理低 16 bit、再处理高 16 bit，保持参考 CRC 的字节顺序。

矩阵引擎：按输出列并行的九个 PE

`mtx9_engine` 面向 CoreMark 的 `N=9` 矩阵操作。A/B 各有 81 个有符号 16 bit 元素，C 为有符号 32 bit 结果数组。矩阵乘法中，PE j 负责一行的第 j 个输出：

$$C_{i,j} = \sum_{k=0}^{8} A_{i,k} B_{k,j}, \qquad j = 0, \ldots, 8.$$

每个 k 将同一个 A 元素送到九个 PE，与缓存的 B 各列元素相乘。乘积先寄存再累加，九次乘积输入后进入 drain 状态，使最后一个乘积完成累加，随后逐项写回该行，再处理下一行。操作数预取、乘积和累加分级，避免把数组选择、乘法和累加全部串在一个组合周期内。

引擎按参考函数的顺序执行 A 加常量、标量乘、向量乘、矩阵乘、位提取乘积累加，最后将 A 减去该常量恢复。向量乘只覆盖 C 的前九项，其余项保留前一遍标量乘的结果，以匹配参考程序随后对整个 C 数组求和的行为。位提取模式将每个乘积按移位 2/掩码 `0xF`、移位 5/掩码 `0x7F` 提取两段并相乘，再进行累加。四遍计算各自进入串行 `matrix_sum`，保留阈值裁剪及相邻元素比较的顺序依赖；每个 16 bit sum 的 CRC 更新另分低、高字节两拍完成。

驱动 `core_bench_matrix()` 将 A/B 各 81 项按 32 bit MMIO 字写入偏移 `0x000/0x200`，在 `0x704/0x708` 写 N 与常量 seed，向 `0x700` 写 start，轮询 `0x70C`，最后从 `0x710` 读取局部 CRC，再调用 `crc16()` 并入调用者传入的 CRC。主存中的 A/B 保持原值，C 的中间结果由引擎内部保存。驱动对非 9 阶矩阵调用软件参考函数。

状态机引擎：两遍扫描与统计归约

`state9_engine` 保存一份最大 1 KiB 的私有数据块。扫描器按照 START、INVALID、S1、S2、INT、FLOAT、EXPONENT 和 SCIENTIFIC 八种状态处理数字 token，维护各状态的 final 与 track 两组 32 bit 计数。数据指针、状态转移和计数增量按参考 `core_state_transition()` 的规则同步推进；逗号作为 token 分隔，NUL 或数据块边界结束扫描。

完整任务依次经过 SCAN1、CORRUPT、SCAN2、UNDO、CRC、DONE。第一次扫描后，每隔 step 个字节用 seed1 的低字节进行异或，跳过逗号；第二次扫描累计到同一组统计中，再以 seed2 执行还原。CRC 阶段按状态编号依次处理 final、track 计数，每个 32 bit 计数按低字节优先更新 CRC，共处理 64 个计数字节。数据数组按四个 byte lane 组织，CPU 可用一次 word store 装入四字节，引擎按扫描地址选择一个 byte lane。

软件将数据打包写入偏移 0x000--0x3FF，依次设置 BLKSIZE、SEED1、SEED2、STEP、CRC_IN，再写 0x700 启动。0x718 返回 busy/done，0x71C 返回最终 CRC。驱动在块长 1–1024 B、step 为正且 seed1=seed2 时选择该路径，其余参数调用软件函数。seed1=seed2 时扰动与还原互相抵消，因此调用者的数据块无需由引擎写回；本次 CoreMark 验证使用的是该参数配置。

链表引擎：用数组影子处理串行指针追踪

list9_engine 用两组 32 项数组保存节点地址与 idx/data16，节点地址作为软件标识保存，不由引擎解引用。scripts/coremark_array_list.c 将当前节点逻辑顺序装入 NODE/VAL，引擎按位置扫描：finder 非负时比较 idx，负数时比较 data16 的低字节。一次 find 每拍检查一个节点；每轮 reverse 只翻转方向位 dir，逻辑位置 k 对应物理位置 k 或 $n-1-k$。

找到节点后，move-to-front 保存待移动项，通过每拍一项的数组移位完成重排。循环同时累计 found、missed 与 retval，最终将 found×4−missed 并入 16 bit 返回值。CoreMark 链表为 31 个节点，seed3=0x66 时该热循环执行 102 轮。

驱动在 NODE 的 0x000 页内偏移和 VAL 的 0x100 偏移写入影子，设置 0x704/0x708/0x70C 的节点数、finder 与循环次数，再写 start。0x710 的 bit2 返回 dir，驱动据此按正向或反向地址读取物理数组，重建软件的节点顺序，并从 0x714 读取 retval。排序、移除/恢复、CRC 遍历及最终链表指针重建继续由软件完成；其中复杂比较函数仍可调用矩阵与状态机驱动。这样，链表热循环整体移入硬件，跨算法的控制关系仍保留在 C 程序中。

本小节对应 core/scsrc/bmu/CRC8.scala、rtl/accel/、scripts/build_coremark_crc.py 与 scripts/coremark_array_list.c；其中 CRC8.scala 的路径相对 rtl/pipeline/。

2.4.2 AMP 启动与 mailbox 协作

私有计算状态与统一程序镜像

pipeline_amp_top 按 CORES 例化 2/4 个 student_top。各 hart 拥有私有 32 KiB IROM、四路 ICache、DCache、数据 BRAM、MMIO 和三个加速器，IROM 装载同一份程序。软件链接的数据窗口均从 0x80100000 开始，ISS 地址生成对 cacheable 数据地址施加 DATA_OFFSET=h*0x8000；IROM 与 MMIO 地址保持原值。每 hart 数据 BRAM 按窗口内低地址索引，因此相同的软件地址落入各自的物理存储。

复位后，各 hart 从相同入口建立栈并清理自己的 BSS。运行到 portable_ini

t() 时，软件读取 0xA0000FF0 的 hart ID：hart0 继续进入 CoreMark 主流程，其他 hart 进入 worker 等待函数。该配置以私有存储承担计算，以共享 uncached mailbox 传递参数和结果。RT-Thread Nano 则使用后文的独立单核配置。

mailbox 初始化、寄存器与访问顺序

mailbox 位于 0xA000F000--0xA000F0FF，分为四个槽，槽 h 的基址为 0xA000F000+h*0x40。pipeline_coremark_mbox 复位后用写口每拍清一个字，完成 64 字清零后置 init_done。各 hart 的复位包含 ~mbox_init_done，因此程序启动时看到完整清零的 GO/DONE 窗口。

表 26: mailbox 槽内字段与协作方向

槽内偏移	字段	作用与写入方
0x00	GO	hart0 最后写 1，发布本次任务
0x04	DONE	worker 最后写 1，发布完成结果
0x08--0x10	ITER / EXECS / SIZE	hart0 设置迭代数、算法掩码和数据分区大小
0x14--0x1C	SEED1–SEED3	hart0 设置该 context 的 seeds
0x20--0x2C	四类 CRC	worker 写 list / matrix / state / final CRC
0x30	ERR	worker 结果字段，成功路径写 0

mailbox 不经过 DCache。每槽有一个写口，任务发布前由 hart0 写参数，GO 发布后由对应 worker 写结果，软件按阶段交接写入职责。参数与 GO、结果与 DONE 均经同一个顺序 MMIO 端口提交，驱动在发布标志前使用编译器 memory barrier 保持访问顺序。读口先寄存字索引，再寄存各槽的 16:1 选择结果，最后做 4:1 槽选择，与桥的两拍读响应对齐。

MULTITHREAD 的启动、并行执行与汇总

1. hart0 在计时开始前准备 context 描述，调用 start_time() 后，依次调用所有 context 的 core_start_parallel()。context0 此时只登记；对 context1 至 context$N-1$，先写 ITER、EXECS、SIZE、seeds，再写对应 GO=1。
2. worker 观察到 GO 后读出参数，在本地数据块初始化 list、matrix 和 state，然后执行自己的 iterate()。传递的是描述字段，数据在 worker 的私有存储中构造。
3. hart0 完成全部释放后进入 core_stop_parallel()。对 context0 执行本地 iterate()，使其与已释放的 worker 重叠运行。
4. worker 计算完成后依次写四类 CRC 和 ERR，最后写 DONE=1，并以 a0=0、

EBREAK 结束。hart0 在处理其余 context 的 stop 调用时轮询对应 DONE，再把结果读回本地 `core_results`。

5. 全部 context 汇总后，hart0 调用 `stop_time()`，由 CoreMark 检查 CRC 并打印一份聚合报告。总迭代数为 context 数乘以每 context 迭代数。

图 22 给出上述过程。worker 的初始化在 GO 之后，因此聚合计时包含 worker 初始化、启动与同步。`pipeline-coremark-full-mt` 的常规仿真总迭代为 200，两核各 100 次、四核各 50 次。另一个 `pipeline-coremark-mc` 入口让各 hart 各自打印 MULTITHREAD=1 报告，属于独立跑分方式。

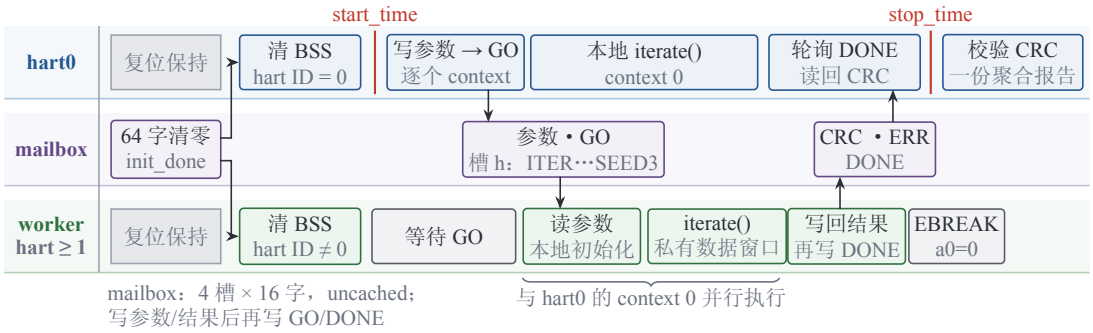


图 22: AMP mailbox 启动、并行执行与结果汇总

对应实现为 `rtl/pipeline/soc/amp/pipeline_amp_top.sv`、`rtl/pipeline/soc/mc/pipeline_coremark_mbox.sv` 与 `scripts/coremark_mt_pipe/core_portme.c`。

2.4.3 RTOS 中断入口与上下文切换

从复位入口到第一个线程

ARES BSP 使用 RT-Thread Nano [10] 3.1.5 的单核调度器，单独链接 RTOS 自检或 MSH 固件。`start.S` 设置 gp、sp 和直接模式 `mtvec=trap_entry`，清除 BSS 后调用 `entry()`，由内核建立调度器、main 与 idle 线程。`rt_hw_board_init()` 启动定时器并设置 `mie.MTIE`。

`rt_hw_stack_init()` 将线程栈按 16 字节对齐，构造与恢复代码一致的 32 字现场：epc 指向线程入口，a0 为线程参数，ra 指向线程退出函数，初始 `mstatus=0x1880` 设置 MPP=M、MPIE=1。`rt_hw_context_switch_to()` 装入第一个线程的栈指针，恢复寄存器，通过 MRET 进入线程并恢复中断使能。

硬件精确进入机器定时器中断

定时器将毫秒计数同步到 CPU 域，观察到新毫秒时置 pending，直至软件 ACK。`mstatus.MIE`、`mie.MTIE` 和 `mip.MTIP` 同时有效后，核停止新发射，让已经发射的

EX、WB、乘除法、LSU 和桥请求继续排空。最后退休指令的 pc_next 持续更新恢复地址；排空期间仍处理分支纠正，进入条件同时排除分支重定向与 Load miss 保护路径。核将后端空闲寄存一拍后进入 trap，向 IFU 发重定向，跳转到 mtvec。

进入时保存该恢复地址到 mepc，写 mcause=0x80000007，将原 MIE 转存到 MPIE 并清 MIE。ECALL 同样进入 trap，但 mcause 为 11，mepc 为 ECALL 指令地址。MRET 跳转至 mepc，并由 MPIE 恢复 MIE。硬件负责精确的控制流边界，通用寄存器现场由 BSP 汇编保存。

trap 栈帧与 C 处理函数

trap.S 在被打断线程的栈上分配 32 个 RV32 字，即 128 B，保存 x1、x4–x31、mepc 与 mstatus，再将 mcause 和 mepc 作为参数传给 rt_hw_trap_handler()。

表 27: RTOS 上下文栈帧（以保存后的 sp 为基址）

字索引 / 字节偏移	内容	恢复含义
0 / 0x00	epc / mepc	中断恢复 PC 或线程续执行地址
1 / 0x04	x1 / ra	函数返回地址
2 / 0x08	mstatus	MRET 使用的机器状态
3 / 0x0C	gp 槽预留	x3 在整份镜像中固定，由启动入口设置
4–31 / 0x10--0x7C	x4–x31	tp、临时寄存器、参数寄存器和被调用者保存寄存器

x0 恒为零；x2/sp 通过保存或更换帧地址以及释放 128 B 帧恢复。该帧布局由本地 cpuport.c、trap.S 和复用的 libcpu/risc-v/common/context_gcc.S 共同使用。

C handler 对 ECALL 增加计数并返回 epc+4，跳过已经处理的指令。定时器中断则先向 0x80200054 写 1 确认 pending，再读取 0x80200050 的毫秒数，计算其与上次服务时间的无符号差值 elapsed。处理函数在 rt_interrupt_enter()/leave() 之间执行 elapsed 次 rt_tick_increase()，定时器中断的恢复 PC 保持原 epc。

pending 表示“有时间变化需要服务”，多个毫秒可以合并为一次 pending。CPU 关闭中断时计时器继续计数，因此按 elapsed 补 tick；新毫秒与 ACK 同拍时硬件置位优先，使随后一次中断继续处理新到时间。内核 tick 频率为 1000 Hz，tick 更新推动时间片和延时队列。

中断中的抢占切换

rt_tick_increase() 更新当前线程时间片并检查到期软件定时器，调度器从就绪

队列选择最高优先级线程。在中断上下文中，`rt_hw_context_switch_interrupt()` 只登记 `rt_interrupt_from_thread`、`rt_interrupt_to_thread` 和 `rt_thread_switch_interrupt_flag`。多个调度请求保留最初的 from，更新最终的 to，实际栈切换集中在 trap 尾部。

C handler 返回后，`trap.S` 先把返回 PC 写入当前帧。若切换标志置位，先清除标志，将当前 sp 写入原线程 TCB 的 sp 字段，再从目标线程 TCB 装入新的 sp；随后统一恢复目标帧的 mstatus、mepc 和通用寄存器，释放栈帧并执行 MRET。没有切换请求时，同一路径恢复原线程。因此，抢占切换改变的是恢复所用的栈帧，无需把旧线程现场搬运到另一个栈。完整流程见图 23。

线程主动阻塞时使用 `context_gcc.S` 的 `rt_hw_context_switch()`：先保存调用者现场，将调用返回地址同时写入帧的 epc 与 ra，保存线程栈指针，再装入目标线程栈并经 MRET 恢复。线程之后重新被调度时，从上下文切换调用之后继续执行。首次启动、线程主动切换和中断抢占由同一帧布局衔接。

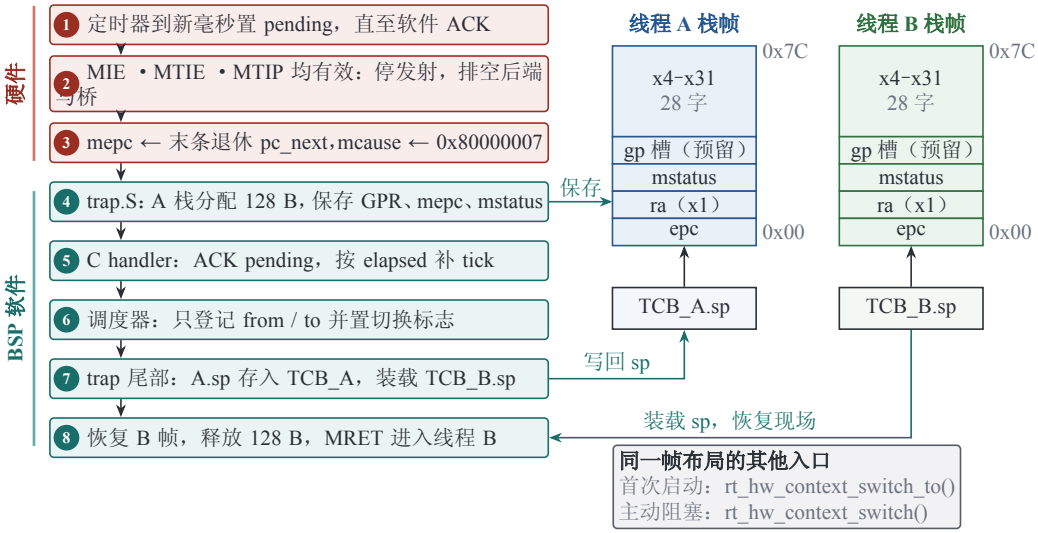


图 23: 机器定时器中断进入与抢占式上下文切换

软件实例与验证对应

自检固件建立两个同优先级、5 tick 时间片的工作线程，在线程内连续忙等、不主动 yield，以检查时间片抢占；同时验证 ECALL 寄存器现场、关中断后的补 tick、`rt_thread_mdelay()` 唤醒和串口输出。MSH 为独立配置，使用 4 KiB shell 栈，通过 UART RX FIFO 与控制台驱动接收命令。这两份固件使用单核机器模式，移植采用非嵌套中断。自检与 MSH 的测试入口及已测结果见第 3.3 节。

本节对应 ARES BSP 的 `start.S`、`timer.c`、`trap.S/trap.c` 与 `cpuport.c`。

第三部分　完成情况及性能参数

3.1　整体介绍

以下汇总截至 2026 年 11 月 3 日的实现、验证记录及待完成项。

3.2　工程成果

3.2.1　机械成果

使用现有 FPGA 平台，未独立设计机械部件。

3.2.2　电路成果

已具备 Mundus 核、I/DCache、片上存储与外设、CRC8 和三类 MMIO 引擎、单核及 2/4-hart AMP 顶层。AMP 为每 hart 私有 IROM、四路 ICache 与数据 BRAM，通过 uncached mailbox 协作。共享 L2 实验线 `pipeline_mc_soc_top` 的 CoreMark 读响应顺序问题待解决。待实现项包括 RV32F、`FENCE.I` 缓存失效及完整特权/调试支持。

3.2.3　软件成果

已具备 CoreMark 及加速镜像构建、ELF/BIN/HEX 转换、单核和 AMP 测试入口，以及独立 RT-Thread Nano 自检/MSH 固件。

表 28: 实现与验证状态

范围	状态
Core / Cache / 加速器	本次 smoke、36 个 cpu-tests、1520 项 DCache 检查、乘除法、三个引擎与五种单核 CoreMark 镜像通过
ICache 安装一致性	`S_FILL` 与 flush/复位同拍的数据/目录不一致已修复；`pipeline-icache-test` 定向与随机检查通过
短复位	流水线与 LSU 各级 valid 已加同步复位；`pipeline-reset-pulse` 的脉宽/相位扫描无陈旧退休或 Store
AMP 多核	本次 2/4-hart 全加速 MULTITHREAD 聚合测试通过；每 hart 私有 IROM、四路 ICache、DRAM 与加速器
Pango 板级	AMP 顶层、文件表和仿真已入库；四核 PDS 结果见第 3.3.5 节，32 KiB/hart 配置的 PnR 待补
RTOS / MSH	本次 RTOS 自检、物理 UART 板级仿真和 MSH 双向串口测试通过；对应 PDS 时序及实板收发待验收
独立视频工程	摄像头/DDR3/HDMI 独立验证已有记录；与 ARES/miniTensor/Web 的闭环集成待完成，方案见附录 C
前身 Mundus	附录 B 保留 Kintex-7、赛题镜像、分支准确率、资源与功耗的历史记录

程序与模块测试的 RTL 版本、命令、镜像与日志哈希见附录 A。

3.3 ARES 验证与性能结果

3.3.1 测试配置与结果记录

本节为 2026 年 11 月 3 日的复测结果。主机使用 Verilator 5.046，单核程序顶层为 `tb_pipeline_isa`，多核为 `tb_pipeline_coremark_amp`。CPU 仿真时钟为 100 MHz，CoreMark 数据规模为 2000 B，seeds 为 0/0/0x66，ISA/ABI 为 `rv32im_zicsr/ilp32`。单核使用默认 4 KiB 四路 ICache 与 8 KiB 两路 DCache；全加速 MT 每 hart 使用 32 KiB 四路 ICache、私有 32 KiB IROM 和 32 KiB 数据窗口。单核镜像使用 `MEM_STATIC`，MT 使用 `MEM_STACK`，各 context 独立运行。

生成核与镜像哈希、复测命令及日志记录见附录 A。

3.3.2 功能验证

表 29: ARES 功能复测结果

项目	结果
Smoke	两次 Store 分别得到 0x55、0x66；FENCE 前后顺序及 EBREAK 停机通过
ISA	36 个 cpu-tests 全部通过，`ISA pass=36 fail=0`
DCache	两路缓存定向与随机测试通过，共 1520 项检查
乘法 / 除法	乘法 21771 个检查周期通过；除法 2347 次操作通过
MMIO 引擎	矩阵 3507 项、状态机 10928 项检查通过；链表 232 组向量通过
ICache	四路缓存定向与随机测试通过，共 2875 次响应核对，含安装周期的 flush/复位
短复位	两种程序在 1–8 拍脉宽、5 个相位下共 60 次复位，均无陈旧退休或 Store
单核 CoreMark	基线、CRC8、CRC8+ 矩阵、CRC8+ 状态机、四类全加速均通过 CRC 与 EBREAK/a0 检查
全加速 MT	2/4 hart 均输出一份聚合报告，所有 context CRC 正确、所有 hart done=1 且 a0=0
RTOS	自检与板级 UART 仿真均输出 `ARES RTOS PASS`；验证 ECALL 现场、抢占、tick 补偿和延时唤醒
MSH	双向串口命令、参数处理、退格/TAB、连续输入、RX FIFO 和终端恢复测试通过

复测入口包括 `pipeline-smoke`、`pipeline-isa`、`pipeline-dcache-test`、`pipeline-mult-test`、`pipeline-div-test`、`unit-mtx9`、`unit-state9`、`unit-list9`、`pipeline-rtos`、`pipeline-rtos-board-sim` 和 `pipeline-rtos-shell-test`。其中

`pipeline-icache-test` 与 `pipeline-reset-pulse` 是针对两项缺陷新增的定向回归，修复前分别复现了 ICache 安装/flush 同拍和短复位两项缺陷。

3.3.3 单核 CoreMark 与加速收益

CoreMark [11] 在 `start_time()/stop_time()` 间读取 CPU 周期计数器，UART 中的 `Total ticks` 是评分使用的计时周期；测试台末尾的 `cycles` 还包含启动、校验和输出等收尾时间。分数按下式计算，打印至小数点后六位：

$$\text{CoreMark/MHz} = \frac{\text{总迭代数} \times 10^6}{\text{Total ticks}}.$$

表 30: 单核 100 次迭代结果（4 KiB 四路 ICache）

镜像	计时周期 Total ticks	仿真总周期	CoreMark /MHz	加速比
基线	44,742,474	44,798,214	2.235012	1.000×
CRC8	38,350,135	38,404,960	2.607552	1.167×
CRC8 + 矩阵	25,363,720	25,416,064	3.942639	1.764×
CRC8 + 状态机	24,117,405	24,169,678	4.146383	1.855×
四类全加速	7,471,566	7,524,378	13.384075	5.988×

四类全加速为 CRC8 指令加链表、矩阵和状态机引擎，相对同次基线为约 5.99 倍。五种镜像的 `crclist/crcmatrix/crcstate/crcfinal` 均为 `e714/1fd7/8e3a/988c`，输出 `CoreMark PASS`，退出时 a0=0。加速变体修改了 CoreMark 热点函数，用于评估软硬件协同收益，不作为 EEMBC 官方分数。

各镜像的复测命令见附录 A。

3.3.4 两核与四核 MULTITHREAD 聚合

`pipeline-coremark-full-mt` 将总迭代数固定为 200，两核各运行 100 次、四核各运行 50 次，hart0 在所有 context 完成后输出单份报告。计时包含 worker 启动、同步及完成等待；它与每 hart 打印一份 MULTITHREAD=1 报告的 AMP 独立跑分方式分别统计。

表 31: 全加速 MULTITHREAD 结果（32 KiB 四路 ICache/hart）

核数	每 context 迭代数	计时周期	仿真总周期	聚合 CoreMark/MHz
2	100	7,100,450	7,188,608	28.167228
4	50	3,569,562	3,709,479	56.029283

所有 context 的 `crclist/crcmatrix/crcstate` 均为 `e714/1fd7/8e3a`，与种子 0/0/0x66 的参考值一致。`crcfinal` 由每次迭代的结果逐次累积，因而取决于每 context 的迭代数：两核每 context 100 次，四个 CRC 与单核 100 次结果相同，`crcfinal` 为 `0x988c`；四核每 context 50 次，四个 context 的 `crcfinal` 均为 `0x0158`。

四核吞吐约为两核的 1.989 倍。两核聚合分数约为单核全加速的 2.10 倍，但两组配置在 ICache 容量（4 KiB 对 32 KiB/hart）、内存分配方式（`MEM_STATIC` 对 `MEM_STACK`）和代码体积选项上均不同，该比值不能直接解释为多核扩展效率；多核扩展比以本表两种 MT 配置为对照，同配置单核基准待补测。

两个聚合镜像均已放入 32 KiB IROM。占用按 ELF 的 `.boot` + `.text` 统计：

表 32: 本次全加速镜像的 IROM 段占用

镜像	已用字节	剩余字节
单核全加速	28,188	4,580
两核全加速 MT	21,688	11,080
四核全加速 MT	21,740	11,028

单核与 MT 构建采用不同链接及代码体积选项，MT 目标固定启用 `--list-engine --matrix-engine --state-engine --small-code`。

3.3.5 Pango 实现与时序结果

现存两组 PDS 报告均为 2026 年 11 月 1 日的 `pipeline_amp_board_top` 四核构建，工具为 PDS 2025.2-SP3-EN （build 231170），目标为 PG2L200H。两组每 hart 数据 BRAM 均为 256 KiB，RTL 此后缩减为每 hart 32 KiB。

表 33: 四核 PDS 布线后时序（每 hart 256 KiB DRAM）

项目	70 MHz 约束	200 MHz 约束
报告时间（11 月 1 日）	16:03:55	17:44:37
Slow corner setup WNS/ns	+0.945	-5.449
Slow corner setup TNS/ns	0	-86357.860
Slow corner recovery WNS/ns	+8.484	-0.573
报告设计状态	全部约束满足	存在约束违例

表 34: 上述两组 PDS 构建的资源占用

资源	70 MHz 构建	200 MHz 构建	器件容量
CLMA	7961	8041	30075
CLMS	1905	1892	9875
APM（grid）	44	40	370
DRM（grid）	308	329	415
DRM36K	304	304	415
DRM18K	8	48	830

DRM grid 与 DRM18K/36K 是报告的不同统计层级，不能直接相加。70 MHz 最差 setup 路径位于矩阵引擎内部；200 MHz 最差路径从 hart3 数据 BRAM 经 Load 返回及旁路通路到乘法器输入，且存在一条 recovery 违例。因此时序瓶颈分布在加速器内部和核内 Load 返回通路两处。

报告来源见附录 A。70 MHz 的满足约束结论属于对应历史网表；当前 32 KiB/hart DRAM 配置的 PnR 和实板 UART 结果待补。

TODO：插入当前 ARES 的 PDS 与实板证据图。
包括与本节配置一致的资源/时序报告截图及实板 UART 输出；每张图标明核数、频率、每 hart 存储容量、工具版本和镜像，历史 70/200 MHz 报告分别标注。

图 24: ARES PDS 及实板验证证据（待补充）

3.3.6 待完成项

表 35: 待完成项与验收内容

项目	验收内容
ICache 上电初始化	验证目标 FPGA 的 valid 初始化及四态仿真的冷启动行为
FENCE.I	增加 ICache 失效及前端清理路径，验证代码更新后的取指
共享 L2	修复慢响应配对和顺序，再检查多核 CRC、done 与 a0
CoreMark	
FPGA 验收	对 32 KiB/hart DRAM 的完整网表执行 PDS、检查全部时序约束并记录实板串口结果
视频/手势闭环	连接独立视频工程、推理与网页控制，补端到端操作及测量结果

已修复的短复位与 ICache 安装/flush 两项缺陷，修复方式分别见第二部分“总体结构”与第 2.2.3 节，复现资料路径见附录 A。

3.4 应用与加速器特性成果

两个应用的最终指标如表 36。所有应用数据均在 ARES-L 五级应用基线上取得，时钟口径按 27 MHz 板级 / 100 MHz 仿真分别标注。

表 36: 两个应用的最终指标

项目	结果
A0 CPU / miniTensor	17 / 103 char/s；0/40 golden；INT8 文本质量下降
H112	140,742 参数 / 152,056 B；20 token/s 输出节流
H88	107,000 参数 / 118,168 B；128 KB loader
QA	10,475 条 / 638,076 B；留出模板 94%（880/936）
手势	287 帧 INT8 100%；板端 8/8；10 Hz；约 0.2 s 更新
每帧命令 / 周期	1,936 条 / 约 1.31 M（100 MHz 13.1 ms；27 MHz 48.5 ms）
b5 整机	CLMA 35%、DRM 63%、APM 9%；27 MHz WNS +12.85 ns
网页	33 单测、15/15 端到端；约 608 kB；240 FPS

优势可以归结为三点：同一颗自研 CPU 通过 CV-X-IF 接一个 miniTensor，就能同时覆盖语言模型与手势 CNN；同一 4×4 阵列既能做卷积，也能把 GEMV 映射为 1×1 卷积；两个应用共用驱动与量化工具链，指标可逐字节/逐 token 对拍。

第四部分　总结

4.1　可扩展之处

后续优先验证 ICache 有效位的上电初始化，再补齐 Cache 与 Load 时机修正、IRQ、AMP 和加速器回归证据，核对目标频率下的 PDS 时序、资源和实板输出。后续扩展包括 `FENCE.I` 失效、共享 L2 响应顺序修正和 RV32F，以及将附录 C 的摄像头缩略图、miniTensor 手势推理与网页控制接入 SoC，形成端到端闭环。每组结果应固定 RTL、参数、程序镜像、工具和测量方法，历史准确率及功耗另行归档。

下一步把命令表回放、8×8 阵列/50 MHz、QAT 与多模态输入纳入同一数据流，并继续在 ARES-L 五级应用基线上部署这两个应用。

4.2　心得体会

高主频设计需要把计算路径、存储器延迟和流水控制放在同一时序框架内考虑。将 BPU 读表流水化后，提前查询和预测保持共同完成请求与响应对齐，维持稳态取指吞吐。数据相关处理结合生产者位置和消费者需求，选择旁路或互锁。

将 AGU 提前到 ISS 可以减少部分 load-use 停顿，但同时引入紧邻基址依赖的约束；Store Buffer 可以减少写入占用，却需要正确处理逐字节读转发、写队列排空和外设顺序。这些优化说明，局部延迟收益必须在完整程序和统一提交语义下检验。

定向测试的价值在收尾阶段尤为明显。程序回归长期通过时，ICache 仍存在一个只在特定周期暴露的缺陷：数据阵列写入只由回填完成状态决定，目录安装却位于复位和 flush 判断之后，若重定向恰好落在安装当拍，缓存会以旧 tag 指向新数据。同样，各流水级 valid 原先不随复位清零，短复位脉冲之后可能退休陈旧指令或写出陈旧 Store。两者都是先编写能稳定复现的测试（安装周期注入 flush/复位、脉宽与相位扫描），再修改 RTL，并将测试保留为回归。这让我们认识到，协议边界和复位行为需要与功能同等对待，并由针对性的测试台持续守护。

验证结果还需要区分配置和统计口径。原生 RV32M 与软件乘除例程具有不同的动态分支行为，预测器及 RAS 收益应在相同代码流和镜像下比较。综合功耗估算、仿真周期与板卡运行时间分别记录。保留配置、日志和照片，使功能、时序及性能结论能够对应到具体工程状态，是处理器持续演进的重要基础。

应用联调最有价值的是五个可复现 bug 和“先测量再优化”：补丁按 0..255 原始字节喂进 int8 引擎、相机固件复制推理序列、裕度循环从 best=0 起、取景框偏移写成 80、剪刀裕度天生约 6；pad/pool 词宽化把帧时间从 4.81 M 降到 2.00

M。宏指令先写单条 LOAD、单条 GEMM2D、单条 CONVGROUP 的单元测试，再上整网。

第五部分　参考文献

[1] Jonathan Bachrach, Huy Vo, Brian Richards, Yunsup Lee, Andrew Waterman, Rimas Avižienis, John Wawrzynek and Krste Asanović. Chisel: Constructing Hardware in a Scala Embedded Language. In *Proceedings of the 49th Design Automation Conference*, pages 1212–1221, 2012.

[2] Andrew Waterman and Krste Asanović, editors. *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. Document Version 20191213. RISC-V Foundation, 2019.

[3] Andrew Waterman, Krste Asanović and John Hauser, editors. *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*. Document Version 20211203. RISC-V International, 2021.

[4] Sparsh Mittal. A Survey of Techniques for Dynamic Branch Prediction. *Concurrency and Computation: Practice and Experience*, 31(1):e4666, 2019.

[5] James E. Smith. A Study of Branch Prediction Strategies. In *Proceedings of the 8th Annual Symposium on Computer Architecture*, pages 135–148. IEEE Computer Society Press, 1981.

[6] David R. Kaeli and Philip G. Emma. Branch History Table Prediction of Moving Target Branches Due to Subroutine Returns. In *Proceedings of the 18th Annual International Symposium on Computer Architecture*, pages 34–42, 1991.

[7] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*, 6th edition. Morgan Kaufmann, 2017.

[8] James E. Smith and Gurindar S. Sohi. The Microarchitecture of Superscalar Processors. *Proceedings of the IEEE*, 83(12):1609–1624, 1995.

[9] Wilson Snyder. Verilator. `https://www.veripool.org/verilator/`.

[10] RT-Thread Development Team. RT-Thread Nano. `https://github.com/RT-Thread/rtthread-nano`.

[11] EEMBC. CoreMark. `https://www.eembc.org/coremark/`.

[12] Qwen Team. Qwen2.5 Technical Report. arXiv:2412.15115, 2025.

[13] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam and Dmitry Kalenichenko. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2704–2713, 2018.

[14] Kyunghyun Cho, Bart van Merriënboer, Dzmitry Bahdanau and Yoshua Bengio. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pages 1724–1734, 2014.

[15] Yann LeCun, Léon Bottou, Yoshua Bengio and Patrick Haffner. Gradient-based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[16] Three.js. `https://threejs.org/`.

[17] Web Serial API. `https://wicg.github.io/serial/`.

附录 A　复现信息

A.1　版本与结果记录

第三部分的程序与模块复测于 2026 年 11 月 3 日完成，使用提交 `bbdc41c` 的 RTL 与本地构建的软件镜像。测试所用生成核 `rtl/pipeline/core/vsrc/build/Mundus.sv` 的 SHA-256 为：

```
2a71b86a9f91797577e3bca1035f7738e48215932981d353edb1fda3572e52f0
```

`docs/technical/results.json` 保存复测命令、参数、二进制与 ELF 哈希、日志摘要及哈希；运行日志位于 `build/docs-review/`。两组 PDS 报告分别为 `results/pipeline_amp_board_top.rtr/.ror`（70 MHz）与 `results/pipeline_amp_board_top(1).rtr/.ror`（200 MHz），其文件哈希与资源摘要同样记录在 `results.json`。短复位与 ICache 安装/flush 两项缺陷的复现数据和修改方案分别见 `docs/pipeline_reset_analysis.md` 第 11 节和 `docs/microarch/pipeline_icache.md`。

A.2　复测命令

单核 CoreMark 各镜像的运行目标如下。它们消费已有镜像；从新检出构建时，先执行 `make coremark-image` 与 `make pipeline-coremark-images`。

```
make pipeline-coremark
make pipeline-coremark-crc
make pipeline-coremark-matrix
make pipeline-coremark-state
make pipeline-coremark-full
```

两核与四核全加速 MULTITHREAD 聚合测试：

```
make pipeline-coremark-full-mt CORES=2
make pipeline-coremark-full-mt CORES=4
```

附录 B　旧版 Mundus 验证资料与历史性能

以下日志、准确率表、板卡照片、资源和功耗图来自旧版 Mundus，配置包括 256 项 BTB、Vivado IP、16 KiB IROM，运行频率为 300–316.667 MHz。

B.1　仿真结果与分析

B.1.1　功能验证

现有报告记录 RV32I 与 RV32M 逐指令测试通过。图 25 展示旧版 RV32M 测试输出，8 项均为 PASS；对应版本的 RV32I 逐条测试截图尚需补充。

```
test list [8 item(s)]: div divu mulh
 mulhsu mulhu mul rem remu
[           div] PASS
[          divu] PASS
[          mulh] PASS
[        mulhsu] PASS
[         mulhu] PASS
[           mul] PASS
```

图 25: 旧版 Mundus：RV32M riscv-tests 测试结果

除逐指令测试外，我们还在相同仿真环境中运行了竞业达提供的 src0、src1、src2 和 withMext 程序镜像。四组程序均执行至 HITGOODTRAP 并正常退出，如图 26 和图 27 所示。日志同时给出了总时钟周期数、提交指令数和 IPC，说明处理器不仅能够完成基础指令测试，也能够连续运行规模更大的赛题程序；其中 withMext 的通过验证了旧版数据通路对 RV32M 乘除法指令的支持。

B.1.2　分支预测器验证与预测收益分析

分支预测差分框架分别以纯 Bimodal/BTB 和 Bimodal/BTB+RAS 两种配置运行同一测试镜像，并逐事件比较 RTL 与 C++ 参考模型给出的方向和目标。在现有各组报告中，两种配置的模型匹配率均为 100%，说明新增的返回指令标记、RAS 更新以及目标覆盖逻辑与参考模型一致。

当 CoreMark 在未使用原生 RV32M 乘除法指令的代码流上测试时，乘除运

```
[/home/redbean/ysyx-workbench/npc/csrc/wave/utils/log.c:18 init_log]
 Log is written to /home/redbean/ysyx-workbench/npc/build/jyd-log.tx
t
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:81 load_img] The
 image is /home/redbean/ysyx-workbench/am-kernels/tests/jyd-tests/bu
ild/src0_jyd_image.bin, size = 1048624
[/home/redbean/ysyx-workbench/npc/csrc/wave/branch_difftest.cpp:348
ensure_inited] branch difftest reference predictor: bimodal
[/home/redbean/ysyx-workbench/npc/csrc/wave/branch_difftest.cpp:365
branch_difftest_set_report_path] branch predictor difftest: ON, pred
ictor=bimodal, report=./build/branch_difftest_report.txt
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:60 welcome] Buil
d time: 15:39:41, Jul 20 2026
[/home/redbean/ysyx-workbench/npc/csrc/wave/cpu.c:171 cpu_exec] HIT
GOOD TRAP
[/home/redbean/ysyx-workbench/npc/csrc/wave/branch_difftest.cpp:728
branch_difftest_dump_report] branch difftest report: ./build/branch_
difftest_report.txt
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:190 performance]
 host time spent = 553773664 us
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:191 performance]
 total clock cycles = 2077949060
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:192 performance]
 total guest instructions = 1417754155
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:193 performance]
 IPC: 0.682285
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:194 performance]
 execution time : 6.561944 s (316.667 MHz)
```

(a) src0

```
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:81 load_img
] The image is /home/redbean/ysyx-workbench/am-kernels/tests/jy
d-tests/build/src1_jyd_image.bin, size = 1048624
[/home/redbean/ysyx-workbench/npc/csrc/wave/branch_difftest.cpp
:348 ensure_inited] branch difftest reference predictor: bimoda
l
[/home/redbean/ysyx-workbench/npc/csrc/wave/branch_difftest.cpp
:365 branch_difftest_set_report_path] branch predictor difftest
: ON, predictor=bimodal, report=./build/branch_difftest_report.
txt
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:60 welcome]
 Build time: 16:23:43, Jul 20 2026
[/home/redbean/ysyx-workbench/npc/csrc/wave/cpu.c:171 cpu_exec]
 HIT GOOD TRAP
[/home/redbean/ysyx-workbench/npc/csrc/wave/branch_difftest.cpp
:728 branch_difftest_dump_report] branch difftest report: ./bui
ld/branch_difftest_report.txt
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:190 perform
ance] host time spent = 532314309 us
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:191 perform
ance] total clock cycles = 1929700196
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:192 perform
ance] total guest instructions = 1304108376
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:193 perform
ance] IPC: 0.675809
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:194 perform
ance] execution time : 6.432333 s (300.000 MHz)
```

(b) src1

图 26: 旧版 Mundus：竞业达 src0 与 src1 程序仿真结果

```
[/home/redbean/ysyx-workbench/npc/csrc/wave/utils/log.c:18 init_log]
 Log is written to /home/redbean/ysyx-workbench/npc/build/jyd-log.tx
t
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:81 load_img] The
 image is /home/redbean/ysyx-workbench/am-kernels/tests/jyd-tests/bu
ild/src2_jyd_image.bin, size = 1048624
[/home/redbean/ysyx-workbench/npc/csrc/wave/branch_difftest.cpp:348
ensure_inited] branch difftest reference predictor: bimodal
[/home/redbean/ysyx-workbench/npc/csrc/wave/branch_difftest.cpp:365
branch_difftest_set_report_path] branch predictor difftest: ON, pred
ictor=bimodal, report=./build/branch_difftest_report.txt
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:60 welcome] Buil
d time: 16:39:15, Jul 20 2026
[/home/redbean/ysyx-workbench/npc/csrc/wave/cpu.c:171 cpu_exec] HIT
GOOD TRAP
[/home/redbean/ysyx-workbench/npc/csrc/wave/branch_difftest.cpp:728
branch_difftest_dump_report] branch difftest report: ./build/branch_
difftest_report.txt
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:190 performance]
 host time spent = 732263419 us
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:191 performance]
 total clock cycles = 2710378734
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:192 performance]
 total guest instructions = 1849099364
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:193 performance]
 IPC: 0.682229
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:194 performance]
 execution time : 9.034595 s (300.000 MHz)
```

(a) src2

```
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:81 load_img] The
 image is /home/redbean/ysyx-workbench/am-kernels/tests/jyd-tests/bu
ild/withoutMext3_jyd_image.bin, size = 1048624
[/home/redbean/ysyx-workbench/npc/csrc/wave/branch_difftest.cpp:348
ensure_inited] branch difftest reference predictor: bimodal
[/home/redbean/ysyx-workbench/npc/csrc/wave/branch_difftest.cpp:365
branch_difftest_set_report_path] branch predictor difftest: ON, pred
ictor=bimodal, report=./build/branch_difftest_report.txt
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:60 welcome] Buil
d time: 17:36:30, Jul 20 2026
[/home/redbean/ysyx-workbench/npc/csrc/wave/cpu.c:171 cpu_exec] HIT
GOOD TRAP
[/home/redbean/ysyx-workbench/npc/csrc/wave/branch_difftest.cpp:728
branch_difftest_dump_report] branch difftest report: ./build/branch_
difftest_report.txt
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:190 performance]
 host time spent = 301390673 us
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:191 performance]
 total clock cycles = 1163342431
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:192 performance]
 total guest instructions = 783162385
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:193 performance]
 IPC: 0.673200
[/home/redbean/ysyx-workbench/npc/csrc/wave/sim.cpp:194 performance]
 execution time : 3.777085 s (308.000 MHz)
```

(b) withoutMext

图 27: 旧版 Mundus：竞业达 src2 与 withoutMext 程序仿真结果

```
[/home/dragonflame/ysyx-redbean/npc/csrc/wave/utils/log.c:18 init_lo
g] Log is written to /home/dragonflame/ysyx-redbean/npc/build/jyd-lo
g.txt
[/home/dragonflame/ysyx-redbean/npc/csrc/wave/sim.cpp:81 load_img] T
he image is /home/dragonflame/ysyx-redbean/am-kernels/tests/jyd-test
s/build/withMext3_rv32m_jyd_image.bin, size = 1048632
[/home/dragonflame/ysyx-redbean/npc/csrc/wave/branch_difftest.cpp:44
8 ensure_inited] branch difftest reference predictor: bimodal
[/home/dragonflame/ysyx-redbean/npc/csrc/wave/branch_difftest.cpp:46
5 branch_difftest_set_report_path] branch predictor difftest: ON, pr
edictor=bimodal, report=./build/branch_difftest_report.txt
[/home/dragonflame/ysyx-redbean/npc/csrc/wave/sim.cpp:60 welcome] Bu
ild time: 17:39:21, Jul 20 2026
[/home/dragonflame/ysyx-redbean/npc/csrc/wave/cpu.c:171 cpu_exec] HI
T GOOD TRAP
[/home/dragonflame/ysyx-redbean/npc/csrc/wave/branch_difftest.cpp:85
9 branch_difftest_dump_report] branch difftest report: ./build/branc
h_difftest_report.txt
[/home/dragonflame/ysyx-redbean/npc/csrc/wave/sim.cpp:190 performanc
e] host time spent = 344388887 us
[/home/dragonflame/ysyx-redbean/npc/csrc/wave/sim.cpp:191 performanc
e] total clock cycles = 518232008
[/home/dragonflame/ysyx-redbean/npc/csrc/wave/sim.cpp:192 performanc
e] total guest instructions = 380344412
[/home/dragonflame/ysyx-redbean/npc/csrc/wave/sim.cpp:193 performanc
e] IPC: 0.733927
[/home/dragonflame/ysyx-redbean/npc/csrc/wave/sim.cpp:194 performanc
e] execution time : 1.636522 s (316.667 MHz)
```

图 28: 旧版 Mundus：竞业达 withMext 程序仿真结果

算需要通过软件例程完成，由于数据的随机性，位移乘法会导致分支预测难以进行。在没有乘除指令的情况下，CoreMark 使用 Bimodal 预测条件分支、跳转和总体准确率分别为 70.26%、74.72% 和 70.75%。该组旧版 RV32M 测试镜像直接使用硬件 `mul/div` 等 RV32M 指令，不再通过软乘除例程模拟同类运算；在关闭 RAS、仍采用 256 项 Bimodal/BTB 的情况下，三项准确率分别达到 92.48%、94.65% 和 92.75%，对比如表 37 所示。

表 37: 旧版 Mundus：CoreMark 的 Bimodal 预测结果对比

CoreMark 代码流	条件分支准确率	跳转准确率	总体准确率
软件乘除例程	70.26%	74.72%	70.75%
原生 RV32M	92.48%	94.65%	92.75%

原生乘除法指令缩短了动态指令序列，并减少软乘除函数调用、返回以及循环分支对 BTB 表项的占用和冲突，使剩余控制流更集中、更容易被 Bimodal/BTB 学习。因此，该组历史代码流的总体准确率较软件乘除版本提高 22.00 个百分点。

在此基础上，表 38 使用相同 RV32M 镜像 (即含有乘除法指令) 进一步比较 RAS 开关造成的变化，从而单独评价 RAS 的收益。

表 38: 旧版 Mundus：Bimodal/BTB 与 Bimodal/BTB+RAS 预测结果对比

测试程序	RAS	条件分支准确率	跳转准确率	总体准确率
CoreMark	关闭	92.48%	94.65%	92.75%
CoreMark	开启	92.48%	98.98%	93.30%
src0 RV32M	关闭	99.39%	99.93%	99.39%
src0 RV32M	开启	99.39%	99.94%	99.39%
src1 RV32M	关闭	97.11%	100.00%	97.31%
src1 RV32M	开启	97.11%	100.00%	97.31%
src2 RV32M	关闭	79.35%	99.96%	82.88%
src2 RV32M	开启	79.35%	100.00%	82.89%
withMext RV32M	关闭	98.77%	99.84%	98.79%
withMext RV32M	开启	98.77%	99.86%	98.79%

CoreMark 中函数调用与返回较为频繁，因此最能体现 RAS 的收益。关闭 RAS 时，7,704,471 次跳转更新中有 412,430 次预测错误；启用 8 项 RAS 后，正确预测数由 7,292,041 增加到 7,626,169，跳转错误数降至 78,302。对应的跳转准确率提高 4.33 个百分点，并使总体准确率从 92.75% 提高到 93.30%。图 29 给出了两种配置下的完整报告。

```
Test Image: /home/dragonflame/ysyx-redbean/am-kern
els/benchmarks/coremark/build/coremark-riscv32-jyd
.bin
Predictor Model: bimodal
CONFIG_BRANCH_BTB_WIDTH: 8
CONFIG_BRANCH_BIMODAL_BITS: 2

[Fetch-side Model vs RTL]
fetch_events:                   310286205
branch_fetch_events:            56435393
branch_dir_mismatch:            0
branch_target_mismatch:         0
branch_full_match_rate:         100.00%
jump_fetch_events:              8402950
jump_dir_mismatch:              0
jump_target_mismatch:           0
jump_full_match_rate:           100.00%
total_predictable_fetch:        64838343
total_full_match_rate:          100.00%

[RTL Prediction Quality]
update_events:                  60914719
branch_update_events:           53210248
branch_full_correct:            49209069
branch_full_accuracy:           92.48%
jump_update_events:             7704471
jump_full_correct:              7292041
jump_full_accuracy:             94.65%
total_full_accuracy:            92.75%
```

(a) 关闭 RAS

```
Test Image: /home/dragonflame/ysyx-redbean/am-kern
els/benchmarks/coremark/build/coremark-riscv32-jyd
.bin
Predictor Model: bimodal+ras
CONFIG_BRANCH_BTB_WIDTH: 8
CONFIG_BRANCH_BIMODAL_BITS: 2
CONFIG_BRANCH_RAS: enabled
CONFIG_BRANCH_RAS_WIDTH: 3

[Fetch-side Model vs RTL]
fetch_events:                   308615566
branch_fetch_events:            56393399
branch_dir_mismatch:            0
branch_target_mismatch:         0
branch_full_match_rate:         100.00%
jump_fetch_events:              8262883
jump_dir_mismatch:              0
jump_target_mismatch:           0
jump_full_match_rate:           100.00%
total_predictable_fetch:        64656282
total_full_match_rate:          100.00%

[RTL Prediction Quality]
update_events:                  60914719
branch_update_events:           53210248
branch_full_correct:            49209069
branch_full_accuracy:           92.48%
jump_update_events:             7704471
jump_full_correct:              7626169
jump_full_accuracy:             98.98%
total_full_accuracy:            93.30%
```

(b) 开启 RAS

图 29: 旧版 Mundus：CoreMark 中 RAS 对分支预测结果的影响

src2 中条件分支事件占比较高，且其 79.35% 的方向/目标预测结果不受 RAS 影响，因此 RAS 对总体准确率的提升仅在四舍五入后表现为 0.01 个百分点。不过，跳转错误仍由 4,756 次减少到 158 次，减少约 96.7%，说明 RAS 对返回地址的预测本身有效，只是其收益被数量更多的条件分支事件稀释。对应结果见图 30。

src0、src1 和 withMext 在关闭 RAS 时的跳转准确率已经分别达到 99.93%、100.00% 和 99.84%，说明采用原生 RV32M 指令后，这些程序中的跳转目标已经较容易由 BTB 预测。RAS 使 src0 的跳转错误由 183 次减少到 163 次，使 src1 的跳转错误由 158 次减少到 154 次；withMext 的跳转准确率则进一步提高至 99.86%。由于三者的基础准确率已经很高，改进反映到总体准确率上均不足以在保留两位小数时体现。

综合来看，该组历史测试中原生 RV32M 指令通过消除软乘除例程，简化了程序的动态控制流，使旧版 Bimodal/BTB 相比软件乘除代码流更容易获得较高准确率；RAS 则是在相同代码流上进一步针对函数返回目标进行优化，不改变条件分支的 Bimodal 方向预测。其收益取决于返回指令的频率及原 BTB 对返回目标的预测质量：CoreMark 的跳转准确率提高 4.33 个百分点，收益最明显；src0、src1、src2 和 withMext 的基础跳转准确率已经接近 100%，因此总体收益较小。这一结果说明，RAS 对调用返回密集型负载具有明确价值，而将其设计为可选配

```
Test Image: /home/dragonflame/ysyx-redbean/am-kern
els/tests/jyd-tests/build/src2_rv32m_jyd_image.bin
Predictor Model: bimodal
CONFIG_BRANCH_BTB_WIDTH: 8
CONFIG_BRANCH_BIMODAL_BITS: 2

[Fetch-side Model vs RTL]
fetch_events:                 789507292
branch_fetch_events:          62064111
branch_dir_mismatch:          0
branch_target_mismatch:       0
branch_full_match_rate:       100.00%
jump_fetch_events:            12321173
jump_dir_mismatch:            0
jump_target_mismatch:         0
jump_full_match_rate:         100.00%
total_predictable_fetch:      74385284
total_full_match_rate:        100.00%

[RTL Prediction Quality]
update_events:                71922517
branch_update_events:         59604280
branch_full_correct:          47296549
branch_full_accuracy:         79.35%
jump_update_events:           12318237
jump_full_correct:            12313481
jump_full_accuracy:           99.96%
total_full_accuracy:          82.88%
```

(a) 关闭 RAS

```
Test Image: /home/dragonflame/ysyx-redbean/am-kern
els/tests/jyd-tests/build/src2_rv32m_jyd_image.bin
Predictor Model: bimodal+ras
CONFIG_BRANCH_BTB_WIDTH: 8
CONFIG_BRANCH_BIMODAL_BITS: 2
CONFIG_BRANCH_RAS: enabled
CONFIG_BRANCH_RAS_WIDTH: 3

[Fetch-side Model vs RTL]
fetch_events:                 789484303
branch_fetch_events:          62062912
branch_dir_mismatch:          0
branch_target_mismatch:       0
branch_full_match_rate:       100.00%
jump_fetch_events:            12318676
jump_dir_mismatch:            0
jump_target_mismatch:         0
jump_full_match_rate:         100.00%
total_predictable_fetch:      74381588
total_full_match_rate:        100.00%

[RTL Prediction Quality]
update_events:                71922517
branch_update_events:         59604280
branch_full_correct:          47296549
branch_full_accuracy:         79.35%
jump_update_events:           12318237
jump_full_correct:            12318079
jump_full_accuracy:           100.00%
total_full_accuracy:          82.89%
```

(b) 开启 RAS

图 30: 旧版 Mundus：src2 RV32M 中 RAS 对分支预测结果的影响

置，可以让资源或时序更敏感的场景继续使用结构更简单的 Bimodal/BTB。

B.2 FPGA 验证

B.2.1 硬件平台说明

验证平台为竞业达 FPGA 试验箱的数字孪生平台，目标器件为 Xilinx Kintex-7，旧版工程使用 Vivado 2023.2 完成综合、实现和 bitstream 生成；当前 ARES 主线为 Pango/PDS。

B.2.2 运行流程说明

旧版 FPGA 验证流程包括：生成对应版本的 SystemVerilog；将处理器和 SoC 文件加入 Vivado 工程；更新 IROM/DRAM 初始化文件；生成或更新 IP 输出产物；执行综合、实现和时序检查；生成 bitstream；在数字孪生平台加载工程；最后观察 LED、数码管和计数结果。

B.2.3 测试用例说明

赛方以 irom.coe 和 dram.coe 提供测试镜像。IROM 镜像包含启动代码、指令测试和控制流程，DRAM 镜像提供初始数据。区域赛测试依次覆盖 RV32I/RV32IM、CSR/trap 和性能程序，LED 用于显示各测试项状态，数码管显示通过条数和运行时间。

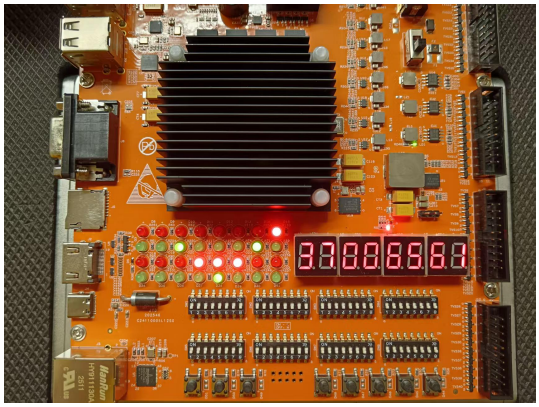
```
memory_initialization_radix=16;
memory_initialization_vector=
00108117,
05010113,
0f5000ef,
670010ef,
0d4000ef,
0000006f,
fe010113,
00112e23,
```

图 31: 旧版 Mundus：withMext IROM 初始化文件的部分内容

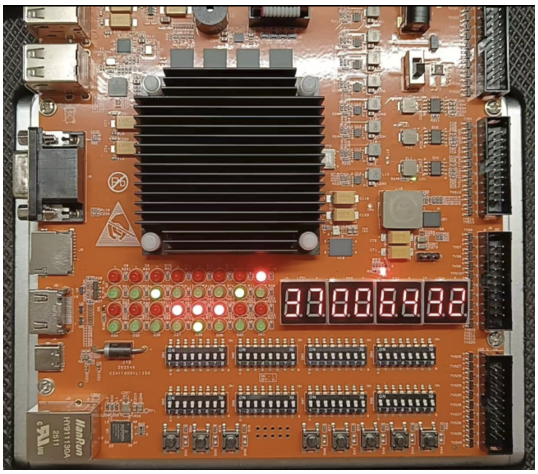
B.3 FPGA 验证结果及分析

B.3.1 FPGA 上板验证结果

集成于官方 SoC 上的九级流水处理器在上板测试中正常运行。多次验证结果表明 RV32I 的 37 条指令、RV32M 的 8 条指令、Zicsr 的 6 条指令及特权相关指令均通过对应赛题测试。



(a) 以 316.67 MHz 运行 src0



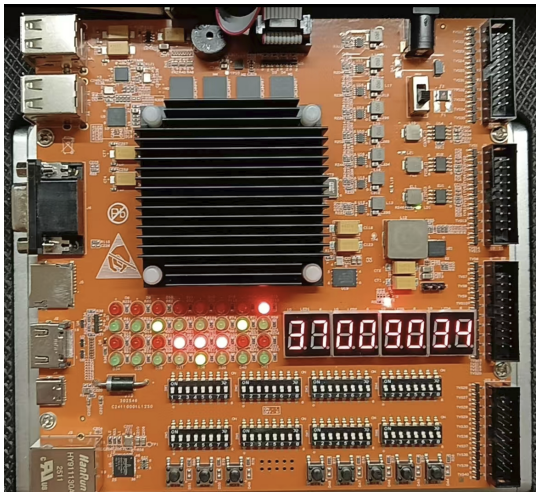
(b) 以 300.00 MHz 运行 src1

图 32: 旧版 Mundus：FPGA src0 src1 运行结果

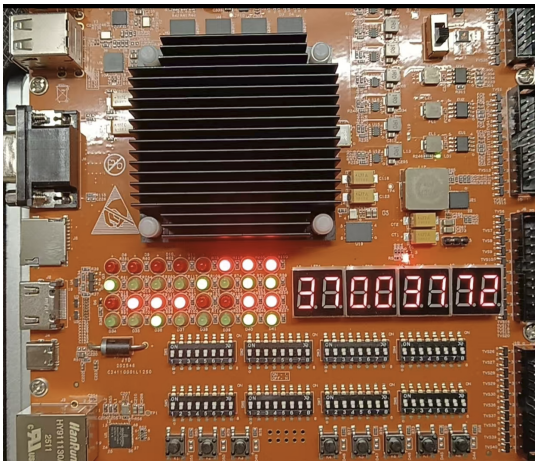
B.3.2 资源使用及面积、功耗分析

根据资源和面积报告可知，由于使用了 BRAM 代替了 IROM 和 DRAM，节省了大量的 LUTRAM 使用和面积占用，故该旧版处理器设计使用的资源较少，LUT、FF 分别占旧 Kintex-7 器件资源的 3% 和 2%。

旧版 Vivado 功耗报告估算芯片总功耗为 0.677 W，动态功耗占比为总功耗的



(a) 以 300.00 MHz 运行 src2



(b) 以 308.00 MHz 运行 withoutMext

图 33: 旧版 Mundus：FPGA src2 与 withoutMext 运行结果

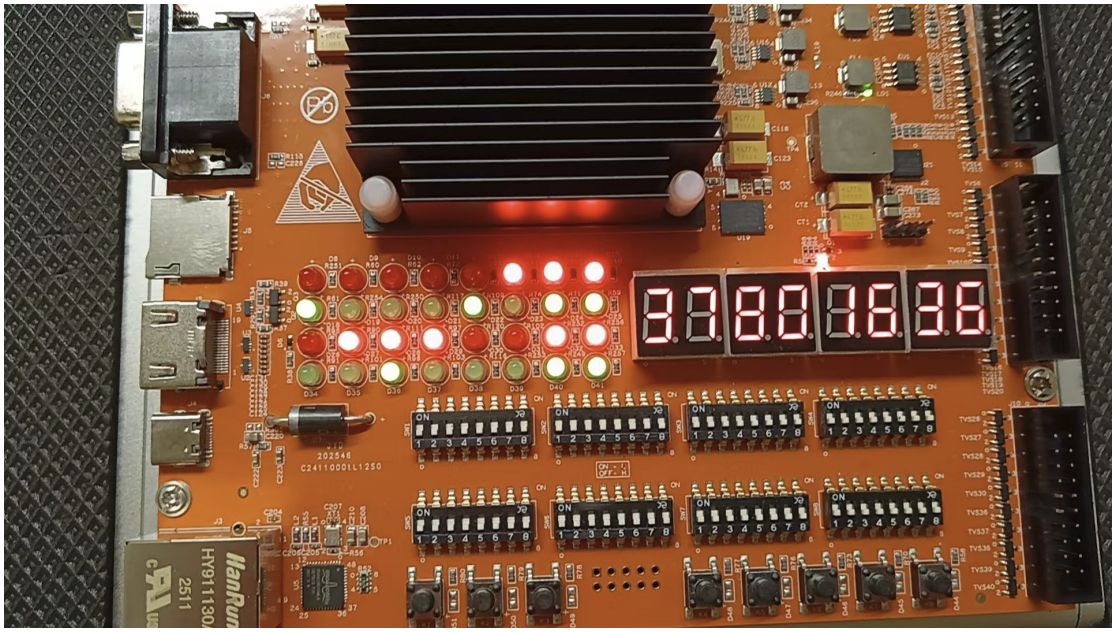


图 34: 旧版 Mundus：以 316.67 MHz 运行 withMext

Resource	Utilization	Available	Utilization %
LUT	6394	203800	3.14
FF	7000	407600	1.72
BRAM	69	445	15.51
DSP	4	840	0.48
IO	76	500	15.20
PLL	1	10	10.00

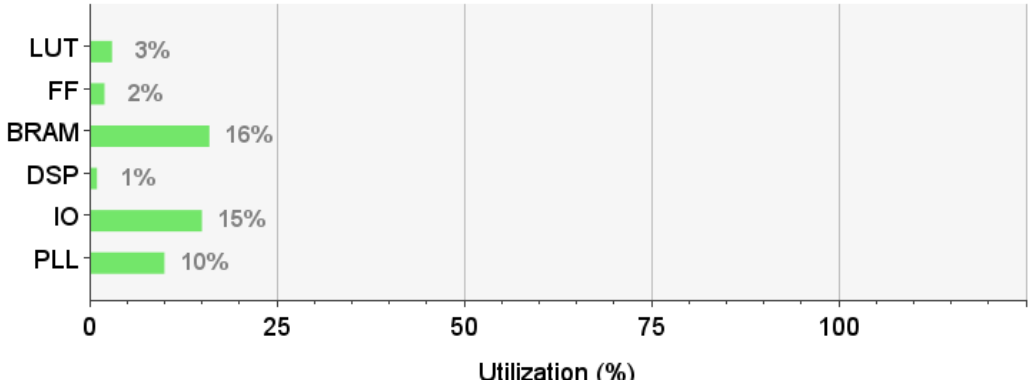


图 35: 旧版 Mundus：资源使用

75 %，静态功耗占比为总功耗的 25%。而在动态功耗中，Clock、Signals、以及 Logic 的占比各自为 19%、29% 和 18%。具体数据如图 37 所示。

B.4 IO 模拟与板级运行一致性

图 38 和图 39 展示旧版 IO 模拟及板级运行。

图 39 记录旧版 60 MHz 配置中 IO 模拟与 FPGA 显示结果的一致性（8.637 s）。

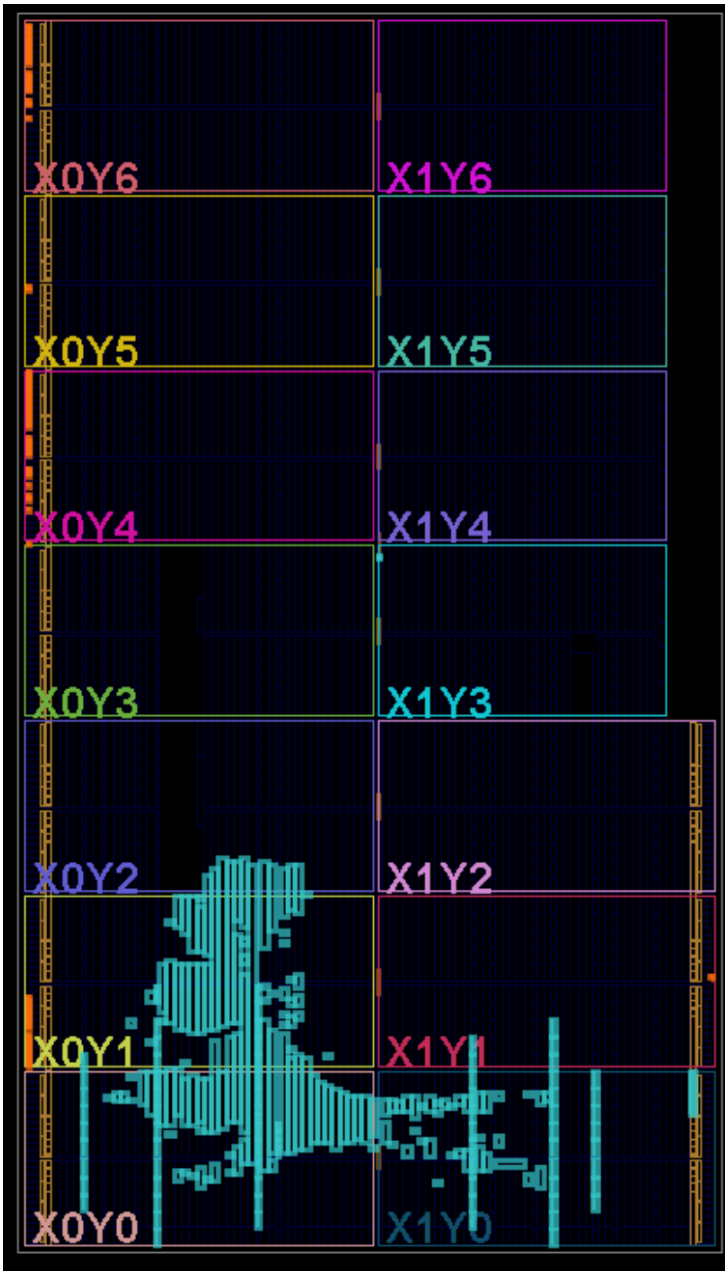


图 36: 旧版 Mundus：综合面积

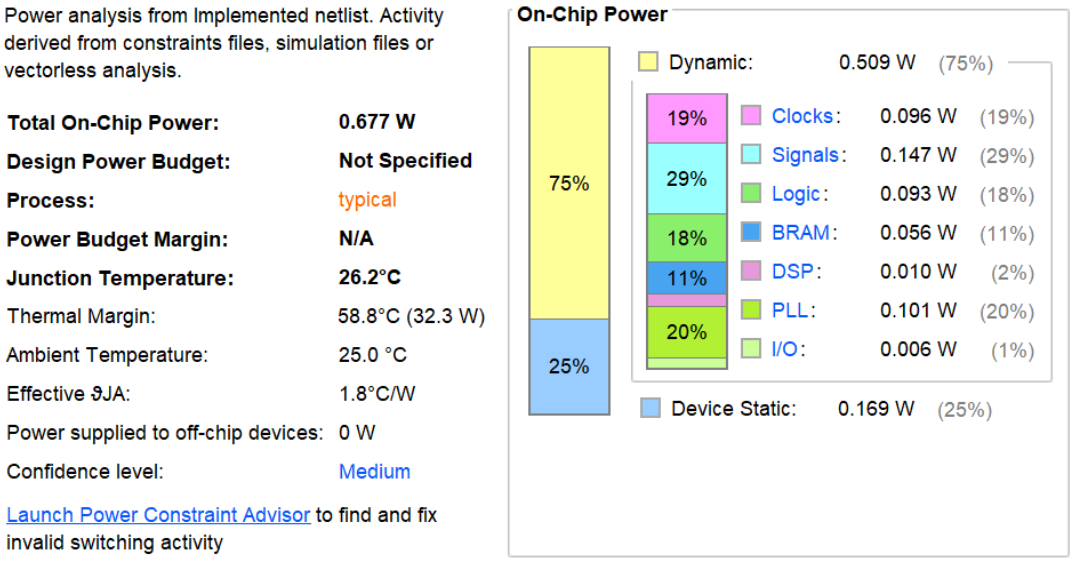
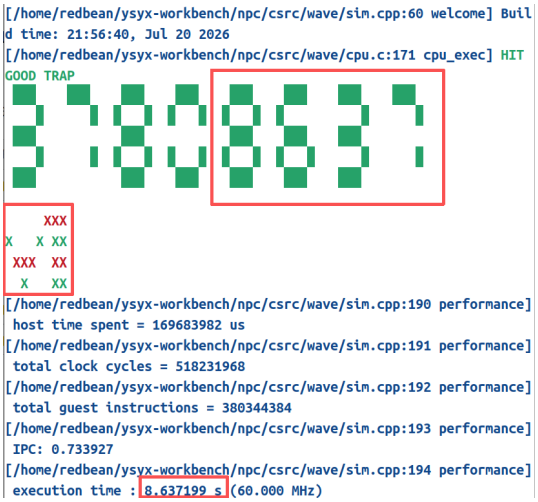


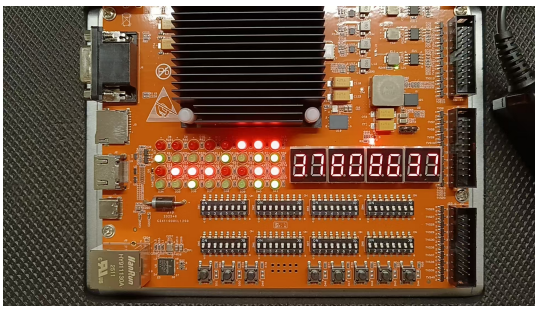
图 37: 旧版 Mundus：功耗占比

```
1  module student_top#(
2      parameter                           P_SW_CNT            = 64,
3      parameter                           P_LED_CNT           = 32,
4      parameter                           P_SEG_CNT           = 40,
5      parameter                           P_KEY_CNT           = 8
6  ) (
7      input                                       w_cpu_clk     ,
8      input                                       w_clk_50Mhz   ,
9      input                                       w_clk_rst     ,
10     input  [P_KEY_CNT - 1:0]                    virtual_key   ,
11     input  [P_SW_CNT  - 1:0]                    virtual_sw    ,
12
13     output [P_LED_CNT - 1:0]                    virtual_led   ,
14     output [P_SEG_CNT - 1:0]                    virtual_seg
15 );
```

图 38: 旧版 student_top IO



(a) 仿真环境 60 MHz 运行情况



(b) FPGA 60 MHz 实际运行情况

图 39: 旧版 Mundus FPGA 与仿真 IO 对比（60 MHz 历史配置）

附录 C　视频采集与手势控制集成方案

OV5640 → DDR3 → HDMI 链路已在独立的 Pango 视频工程 `video_fb_top` 中实现；缩略图 MMIO、miniTensor 手势推理及网页控制尚未接入 ARES，端到端闭环待验证。本附录记录该集成方案。

C.1　OV5640 摄像头视频通路

OV5640 通过 DVP 8 位接口输出图像。FPGA 先通过 SCCB 总线完成传感器上电时序和寄存器配置，将输出格式设置为 RGB565；DVP 接收模块对 8 位字节流做确定性配对，恢复 16 位像素，并在 HREF 低电平期间复位字节相位，避免行长变化或毛刺造成奇偶字节错位。像素处理时钟由 DVP 时钟二分频得到，数据经过三级流水后送入后续处理，以匹配同步存储器和跨时钟域接口的时序。

该方案将视频通路分为缩略图和整帧两条支路，如图 40所示。缩略图支路在 1280×720 有效画面中取中心 640×640 窗口，按 20×20 像素单元做 4:1 盒式滤波，降采样为 32×32 RGB565 图像，并通过双 bank 和异步 FIFO 写入片上 BRAM；方案中由 SoC 通过 MMIO 窗口读取当前帧，供 miniTensor 手势识别使用。整帧支路保留完整 1280×720 图像，经 DDR3 双缓冲后由 HDMI 以 720p30 输出，用于实时预览和演示。

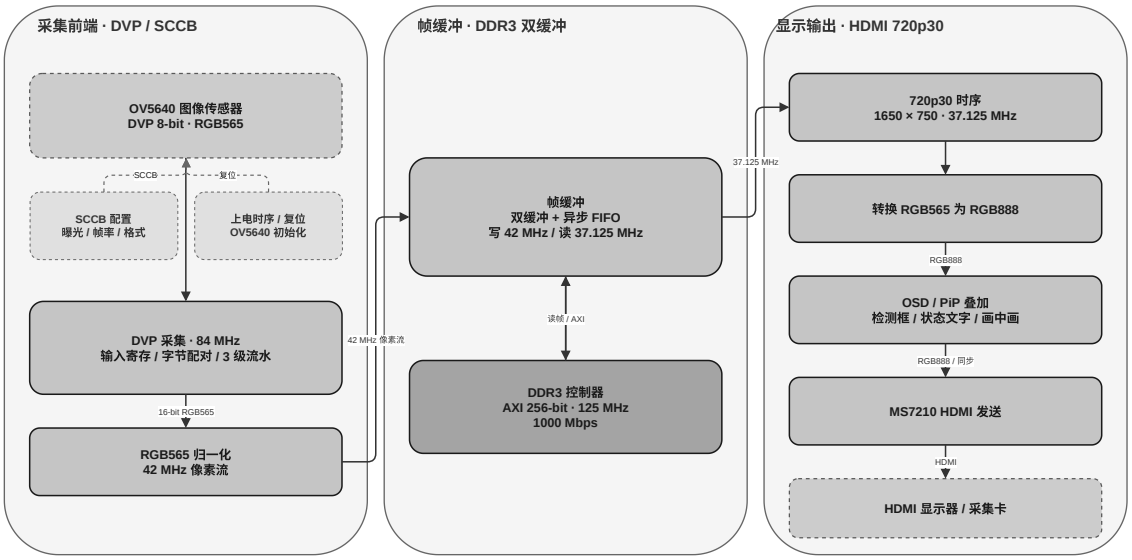


图 40: OV5640 视频集成方案

C.2 手势识别与网页演示控制

该方案拟把摄像头前的手势转换为网页演示控制指令，形成从手势识别、串口传输到网页状态机与视觉反馈的闭环。方案中的硬件端在 FPGA 上完成图像采集、固定取景区裁剪和 32 × 32 灰度归一化，由 miniTensor 上的轻量 INT8 卷积网络输出手势类别与置信度；固件通过连续多帧一致性和置信度门限抑制抖动，再把确认后的手势转换为一行 ASCII 指令，经 115200 8N1 串口发送给上位机。硬件端从图像采集到串口发送的通路如图 41所示。

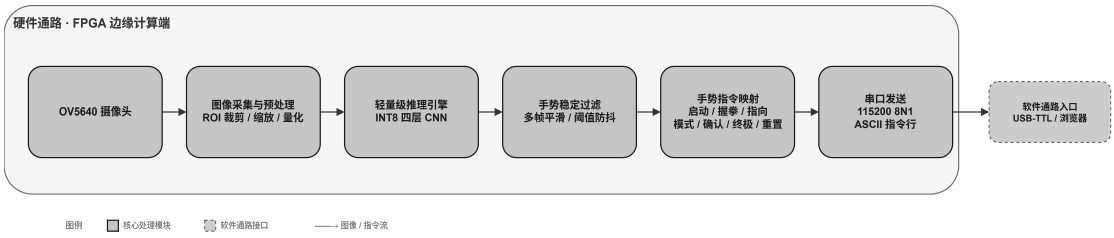


图 41: 手势识别与网页控制的硬件方案

方案中的上位机 Web 控制器采用 Web Serial API 和 Three.js。串口字节流先经过行解析和指令解析，再由冷却/去重逻辑过滤重复手势，进入“基础状态 + 瞬时叠加层”的状态机；状态机只负责逻辑转移，效果层根据状态事件驱动粒子 Logo、全息球、激光扫描、能量波和 HUD 信息层等视觉元素。该分层使协议、状态逻辑和渲染相互独立，便于单元测试和现场调试；无硬件时可用键盘、调试面板或自动演示模式模拟指令。软件端从串口接入到全屏展示的通路如图 42所示。

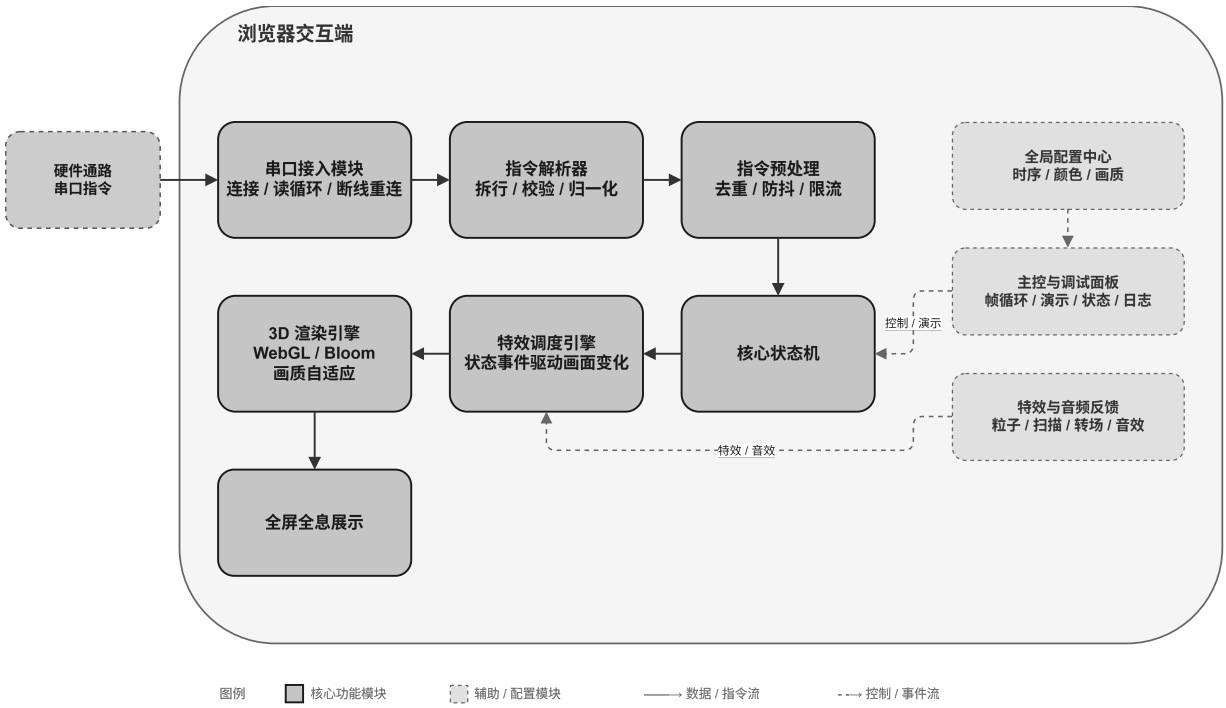


图 42: 手势识别与网页控制的软件方案

附录 D　应用与加速器复现信息

应用代码、模型、权重、287 帧数据集与上位机源码分别在 `5-stage-baseline/` 与 `ares-web-controller/`。主要复现命令：

```
cd 5-stage-baseline && make yolo # golden diff 0
cd 5-stage-baseline/yolo_gesture && make test
cd 5-stage-baseline/llm && bash tools/run_a0.sh
cd ares-web-controller && npm test
```

口径：H112 152,056 B 用 512 KiB 窗口，H88 118,168 B 用 128 KB loader；20 token/s 是输出节流，QA 的 5 tok/s 是 4 字符/token 的显示口径；相机整机 WNS −3.83 ns、3 条时钟未约束，结论为功能验证通过、时序待收敛。